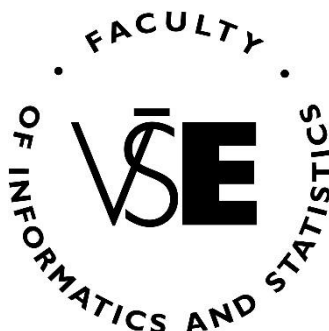


Prague University of Economics and Business

Faculty of Informatics and Statistics



Detection and Explanation of Misinformation Patterns in Fact-checked Claims Using LLMs

BACHELOR THESIS

Study programme: Data Analytics

Author: Richard Mládek

Bachelor Thesis Supervisor: Ing. Kateřina Haniková

Prague, June 2025

Acknowledgement

I would like to express my gratitude to the supervisor of this thesis, Ing. Kateřina Haniková, whose expertise and detailed feedback had an invaluable impact on this work.

I want to thank my family; their endless support and patience made this thesis possible.

I am also grateful to the class of Data Analytics for being the best classmates I could have asked for, and to all my friends who never give up on me, even when I am writing a thesis for a month straight and do not respond to their messages.

Abstract

This bachelor's thesis explores automated approaches to combating misinformation. It proposes a method for modeling argumentation in fact-checked claims using entity graphs and large language models (LLMs).

The source dataset consists of 50 fact-check reports from the PolitiFact website. The argumentation in each report was analysed during an annotation campaign conducted by a team from VŠE. Annotators classified misleading claims based on predefined misinformation patterns in a *Catalogue of Misinformation Patterns* and created brief summaries of the argumentation in each fact-check report.

The automated misinformation pattern detection approach proposed in this thesis achieved an exact agreement with annotators in 46% of reports, with an additional 36% of cases where the model proposed partially matching misinformation patterns. Evaluation of a wide selection of LLMs demonstrated that the *Gemini 2.5 Pro* model, created by Google, performed best in argumentation modeling.

The experiments revealed that some examined claims are more challenging to model automatically than others. Based on this observation, a set of parameters that determine the complexity of a fact-check report is proposed. This metric has the potential to serve as an indicator of how confident we can be in an LLM's automated decision.

To improve the interpretability of argumentation in fact-check reports, a method for automatically sketching entity graphs using LLMs is presented. Preliminary experiments indicate that this approach shows promise, with automatically generated graphs often approaching the quality of those created manually by human annotators.

This thesis demonstrates that it is possible to model argumentation within fact-check reports fully automatically. The proposed approach has the potential to be used for evaluating the quality of fact-check reports, which could enable fully automated refutation of examined problematic claims in the future. Additionally, it improves the interpretability of argumentation in fact-checked claims.

Keywords

Fact-checking, argumentation, entity graphs, large language models, misinformation, interpretability

Abstrakt

Tato bakalářská práce se zabývá automatickými metody boje proti dezinformacím a navrhuje způsob, jak pomocí velkých jazykových modelů (LLMs) automaticky detekovat a modelovat argumentační vzory v zavádějících nebo nepravdivých výrocích, které jsou předmětem fact-checkových reportů.

Jako zdrojový dataset je použit soubor 50 fact-checkových reportů z webu *PolitiFact*. Tento dataset byl vytvořen v rámci anotační kampaně týmem anotátorů z VŠE. Anotátoři analyzovali argumentaci v každém reportu, klasifikovali hlavní dezinformační vzory na základě předem připraveného katalogu dezinformačních vzorů a vytvořili krátké vysvětlení argumentace ve fact-checkovém reportu.

Výsledky automatické detekce dezinformačních vzorů ukazují, že modelování argumentace nejlépe zvládá model *Gemini 2.5 Pro* od společnosti Google, který dosáhl přesné shody s anotátory ve 46 % případech a v dalších 36 % bylo dosaženo částečné shody.

Experimenty v práci dokazují, že některá zkoumaná tvrzení jsou náročnější automaticky modelovat než jiná. Na základě tohoto poznatku je navrhnout soubor parametrů, na kterých závisí *komplexita fact-checkového reportu*. Tato metrika má potenciál být ukazatelem toho, nakolik si můžeme být jisti rozhodnutím velkého jazykového modelu při modelování argumentačních vzorů.

V rámci zlepšení vysvětlování argumentace fact-checkových reportů je v práci nastíněn postup, jak automaticky načrtnout entitní graf pomocí LLMs. Předběžné experimenty ukazují, že grafy které zachycují jádro argumentace lze automaticky generovat.

Tato práce dokazuje, že je možné modelovat argumentaci v rámci fact-checkových reportů zcela automaticky. Navržený postup má potenciál být využit pro vysvětlování dezinformací a pro evaluaci kvality fact-checkových reportů.

Klíčová slova

Fact-checking, argumentace, entitní grafy, velké jazykové modely, dezinformace, interpretabilita

Contents

Introduction	7
1 The Problem of Misinformation	9
1.1 What is fact-checking?	10
1.2 Misinformation Patterns	11
1.2.1 Examples of Misinformation Patterns	12
2 Machine Intelligence in Fact-Checking	16
2.1 Claim detection	16
2.1.1 Verifiability	16
2.1.2 Priority: Determining check-worthiness of claims	17
2.1.3 Similarity: Identifying repeated claims	17
2.1.4 Automatic Claim Detection Approaches	18
2.2 Gathering Relevant Resources & Verdict Determination	19
2.3 Text Summarization	20
2.3.1 Extractive summarization	21
2.3.2 From Abstractive Summarization to Large Language Models	21
2.3.3 Summarization in Fact-Checking	23
2.4 Summary and Outlook	24
3 Annotation Campaign	25
3.1 Methodology of Annotation	27
3.2 Annotation Campaign Results	28
4 Automatic Classification of Misinformation Patterns	31
4.1 Selecting Fact-check Reports for Development	31
4.2 Selecting Suitable Large Language Models	32
4.3 Evaluating LLM Consistency	35
4.4 Fact-Check Report Complexity	38
4.5 Prompt Refinement	39
4.6 Main Experiment	43
4.6.1 Experiment Overview	43
4.6.2 Experiment Evaluation Approach	44
4.6.3 Experiment Results	46
4.6.4 Misinformation Pattern Relatedness	48
5 Automatic Explanation of Misinformation Patterns	51
5.1 Automatic Explanation Generation	51

5.2 Automatic Entity Graph Generation.....	52
5.2.1 Examples of LLM-Generated Graphs	53
6 Conclusions	58
List of references	60
Appendices	I
Appendix A: Politifact Dataset.....	I
Appendix B: Automation Misinformation Classification Experiment results	I
Appendix C: Source Code of Generated Entity Graphs	I
Appendix D: Github Repository Snapshot	I

Introduction

Recent advancements in machine learning technology, especially generative artificial intelligence, have made it ever so simple to create misleading content. Misinformation is especially prominent in online spaces where it can be distributed quickly, causing significant harm. On the other hand, the same modern approaches can also be employed to monitor and counter the spread of false information. This thesis explores how automated methods, and especially large language models, can be utilized to combat misinformation.

Motivation and Objectives

The motivation for choosing this topic was my conviction that the proposed work has the potential to make a tangible impact in the real world. The goal is to create an experiment with LLMs that can automatically detect misinformation patterns in fact-checked claims and explain them. The proposed approach can serve as a practical blueprint for automatically analysing and interpreting fact-check reports and as a building block on the path toward automatically assessing the truthfulness of claims.

Research Questions

The foundation for this work comes from previous research at VŠE, especially representing argumentation behind fact-checked claims using entity graphs (Haniková, 2025). The core focus of this thesis is on verdict argumentation elements, which are based on a pre-established catalog of misinformation patterns. Building on initial experiments that explored the possibility of creating automated summaries that contain the argumentation's core, and to identify argumentation patterns automatically (Haniková et al., 2024), the main research questions are defined as:

RQ1: Can misinformation patterns (as defined by the argumentation patterns catalog) present in fact-check reports be automatically detected?

RQ2: How consistent is the modeling of argumentation in fact-check reports provided by LLMs?

RQ3: Is it possible to automatically generate a model or graph of the argumentation in fact-check reports?

Methodology

The theoretical part of the thesis is based on the method of research and analysis of existing solutions in the field of automated fact-checking. The research also addresses the fundamentals of machine learning and large language models, which are described in the context of the timeline of advancements in automated text summarization methods.

In the first part of the practical section, it is described how the source dataset of 50 fact-checked claims for this thesis was collected and annotated. The results from the annotation campaign are analysed to understand the level of agreement between annotators and the distribution of misinformation patterns selected.

The practical section begins with an experiment that evaluates the consistency of responses from a broad range of LLMs across multiple iterations when a baseline prompt is kept constant. This experiment helps identify the most suitable LLMs for the task and also provides preliminary results for RQ2.

To address RQ1, the main experiment is conducted. A prompt to identify the core misinformation pattern present in a fact-check report is iteratively developed using a selection of fact-checked claims. The refined prompt is then applied to the collected dataset, querying the LLMs via an API service. Each LLM is prompted five times for each fact-check report to gather a broader range of responses and assess consistency. The results are compiled into a dataset and compared to the annotation campaign, with the LLM's decisions categorized into three groups based on their level of agreement with the annotators.

The final section aims to answer RQ3. A second prompt is iteratively developed. It is used in a 2-shot approach, incorporating the output from the first prompt to generate Python code. The code creates entity graphs of the fact-check argumentation. A selection of automatically generated graphs is then compared to examples from the annotation campaign. This comparison assesses the feasibility of the solution and answers RQ3.

Thesis Structure

The thesis is structured as follows. Chapter 1 introduces the problem of misinformation and explains the argumentation modeling techniques employed. Chapter 2 provides theoretical grounding by analysing fact-checking, reviewing existing automated approaches, and introducing the fundamentals of machine learning and large language models. Chapter 3 describes the annotation campaign during which the source dataset was collected and annotated. Chapter 4 presents the main experiment of automatically detecting misinformation patterns, with a comprehensive evaluation of multiple approaches and LLMs. Chapter 5 addresses interpretability by proposing methods to create argumentation graphs automatically.

1 The Problem of Misinformation

In recent years, there has been a sharp increase in the volume of content shared online. Businesses and governmental administrations are becoming increasingly digitalized, and millions of individuals are beginning to use the Internet each year (*World Bank Open Data*, n.d.). With the growing influence of the Internet on daily life and ultimately on the global scale, it is increasingly important to monitor the information disseminated online. A single piece of convincing misinformation can be rapidly propagated worldwide through social media, email, or other communication channels within hours. A recent example is the COVID-19 pandemic, during which numerous hoaxes and false claims were circulated globally, undermining public trust in health measures such as restrictions and vaccination. *Misinformation regarding the incidence rate, prevalence, and spread of the virus has contributed significantly towards the complacent attitude of people to this crisis* (Ferreira Caceres et al., 2022).

False information on the Internet can also shape public opinion across a wide range of topics. One such example is the degree of public acceptance of the fact that our planet is warming and that global warming is occurring.

A major factor contributing to the increased volume of content posted on the Internet has been the adoption of large language models (LLMs). While language models have existed since the 2010s, the most significant turning point in their widespread use arguably occurred with the release of ChatGPT in November 2022. This model enabled end users to generate textual responses based on text input (prompts). The technology sparked a surge of interest in artificial intelligence (AI), leading to the release of numerous models by various companies. The most prominent players in the development of foundation models¹ currently include US-based companies such as OpenAI, Anthropic, Google, and Meta, as well as international organizations like the Chinese firm DeepSeek and the French research lab Mistral, among others. LLMs are being utilised across various fields, with computer science and programming currently being the most prominent areas of application.

One of the fundamental questions is whether artificially generated text can be reliably distinguished from text authored by humans. This question lies at the core of the Turing Test, long regarded as a benchmark for assessing machine intelligence. However, recent studies have demonstrated that LLMs are now capable of producing text that is virtually indistinguishable from human writing in randomized, controlled, and pre-registered Turing tests on independent populations. *The results constitute the first empirical evidence that any artificial system passes a standard three-party Turing test* (Jones & Bergen, 2025).

Various automated approaches, including machine learning-based classifiers, have been proposed to detect machine-generated text, yet the results remain inconsistent and

¹ Large models which require vast compute power and infrastructure to be trained

inconclusive. In my assessment, while detection may be feasible when targeting outputs from a specific, static model, the continual release of new models, along with fine-tuning capabilities and stylistic variability, makes the task of reliably determining whether a given text was written by a human or an LLM increasingly untenable. If the method of text generation cannot be reliably assessed, efforts to combat online misinformation must instead prioritize evaluating the factual accuracy of the content itself.

1.1 What is fact-checking?

Fact-checking is the task of assessing the truthfulness of claims made by public figures such as politicians, pundits, etc. (Vlachos & Riedel, 2014). In essence, it is a binary classification task, where the goal is to determine whether a given claim is true or false. In practice, however, the task is far more complex. Statements are often not simply true or false; they may be partially true, misleading, or fall into various degrees of accuracy. Consequently, a wider range of categories is often needed to capture the subtleties that lie between the basic binary labels.

There are several websites and organisations that specialize in fact-checking statements, typically made by politicians or shared through social media. Their analyses are published as articles known as fact-check reports. Among the most prominent platforms are *PolitiFact*², *FactCheck.org*³, or local initiatives such as the Czech organisation *Demagog*.⁴ Each of them usually has a predefined set of labels used to classify a statement into a category. For the purposes of this thesis, we have decided to work with reports from PolitiFact. Their classification includes categories like *True* and *False*, intermediate ones such as *Mostly True*, *Half True*, and *Mostly False*, and even labels for extreme cases like *Full Flop* and *Pants on Fire*.

A fact-check report is typically centered around a specific claim. A claim is usually a statement or a short piece of text, such as a quote from a prominent politician or public figure, or other content being shared online, including messages or social media posts. The fact-check report is compiled by an individual journalist or a team of reporters. It consists of textual explanations outlining what is inaccurate or misleading about the original statement, supported by references to reliable sources. The analysis aims to evaluate the truthfulness of the original claim. For more complex cases, the report can be several pages long, which may make the reasoning challenging to follow. To address this, a summary section titled *If Your Time Is Short* has been introduced recently to present the key insights in a more digestible format.

² <https://www.politifact.com/>

³ <https://www.factcheck.org/>

⁴ <https://demagog.cz/>

1.2 Misinformation Patterns

Previous research has demonstrated that fact-checked claims can be modeled using graph-based representations (Haniková, 2025). This section serves as an introduction to this approach. The aim of this thesis is not to conduct a graph or ontological analysis of claims (which is the subject of other research), but rather to explore whether LLMs can automatically perform certain tasks in this domain. Specifically, we investigate whether LLMs can detect misinformation patterns and subsequently explain their reasoning.

*PURO Modeler*⁵ was used for the initial graph-based representation of fact-checked claims. This tool was originally developed to draft ontologies, but current research explores its potential for modeling fact-checked claims and identifying what is wrong with them. PURO Modeler provides a graphical interface where users can create models using basic entity graph primitives: objects (particular objects or individuals), types (classes), relationships (connections between entities), attributes (quantitative properties), and values (of attributes) (Dudáš, Svátek, et al., 2016). This straightforward approach makes it accessible for modeling argumentation structures without requiring deep technical knowledge of ontological languages.

By modeling fact-checked claims into graph structures, two types of argumentation elements have been discovered. First, verdict argumentation elements identify what is wrong (or right) in a claim and in what way - these are classified as misinformation patterns and are tied to entity graphs. For example: *misleading relationship*, *exaggerated value*, *false object*, etc. Second, auxiliary argumentation elements capture parts of the fact-checker's argumentation that do not establish a verdict but provide important context. For example: *presumed justification* or *denial justification*. These elements help identify recurring patterns in how misinformation is constructed and how fact-checkers refute false claims.

Verdict argumentation elements form the core focus of this thesis. We refer to them as misinformation patterns because they identify what is specifically wrong in a claim. They represent the various ways in which claims can be misleading or false, ranging from incorrect relationships between entities to false attributes or missing context. Therefore, the terms "verdict argumentation element" and "misinformation pattern" are used interchangeably throughout the thesis.

The following section presents examples of misinformation patterns and demonstrates how we can capture fact-checked claims as entity graphs.

⁵ Available at <https://puromodeler.vse.cz/>

1.2.1 Examples of Misinformation Patterns

Example 1: Joe Biden Signing Documents

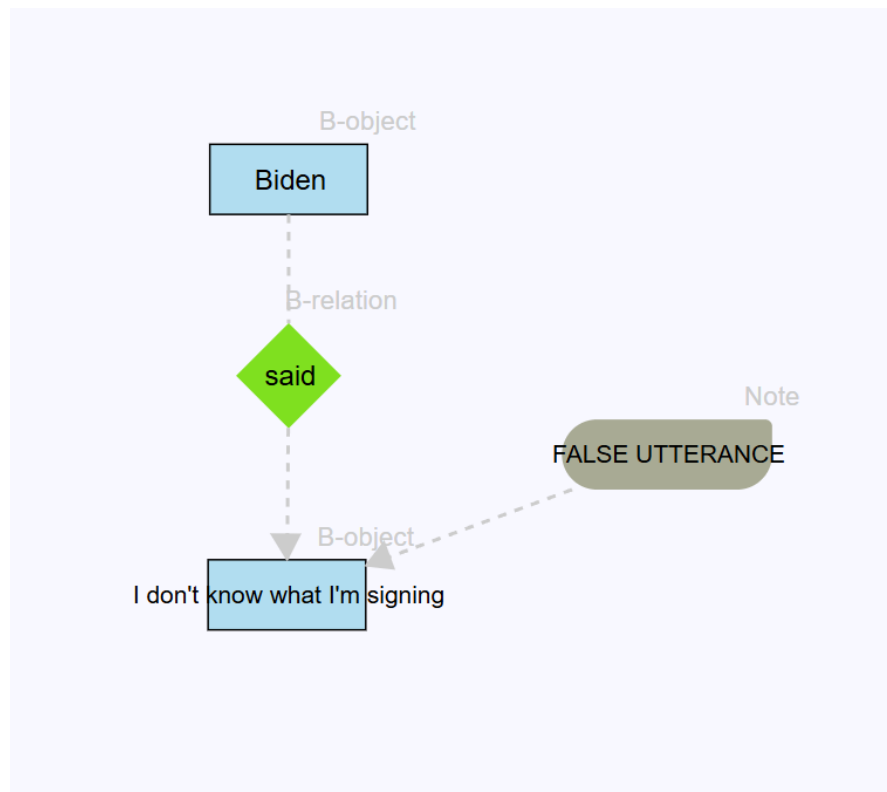
The first example of a misinformation pattern is presented in Figure 1.1. The claim states that *Joe Biden said 'I don't know what I'm signing' while signing a document.*⁶ This claim was modeled using the PURO Modeler in the most straightforward manner possible. In the graph representation, Joe Biden is represented as an entity who allegedly made the statement "I don't know what I'm signing." The relationship between Biden and this statement was classified as *FALSE UTTERANCE*, as the video contains no evidence of him saying these words.

Based on the annotator's analysis, this claim contains a misinformation pattern. The utterance attribution is *false* because experts were unable to verify what Biden actually said while signing the document, as he was wearing a COVID mask at the time. The audio was unclear, making it impossible to confirm whether he uttered this statement. Therefore, this represents a *FALSE UTTERANCE* - a misinformation pattern where statements are falsely attributed to entities such as public figures without verifiable evidence.

⁶ Full fact-check report available at <https://www.politifact.com/factchecks/2021/jan/29/facebook-posts/no-evidence-biden-said-i-dont-know-what-im-signing/>

Figure 1.1

The "document signing utterance" fact-check report entity graph



Source: Author

The entity graph maintains a high level of abstraction to clearly show what makes this claim problematic. Rather than getting lost in details, the model captures the essential issue: an utterance falsely presented as something Joe Biden said. This approach helps explain both why the claim was labeled as false on PolitiFact and what is problematic in the claim's argumentation.

Example 2: Fire Drone Video

The second fact-check report we will examine involves a post on a social media platform X, which features a video allegedly showing a fire drone starting fires in a forest.⁷ The full claim reads *Here is the fire drone video that Twitter, Google, and YouTube didn't want us to see*. The fact-check cites a spokesperson of the California Department of *Forestry and Fire Protection* who explains that the video actually shows a helicopter and that it is not something secret but rather a standard practice called a controlled burn.

⁷ Full fact-check report available at <https://www.politifact.com/factchecks/2020/sep/16/tweets/twitter-video-does-not-show-drone-starting-west-co/>

When we analyse this claim, we can identify two false assertions:

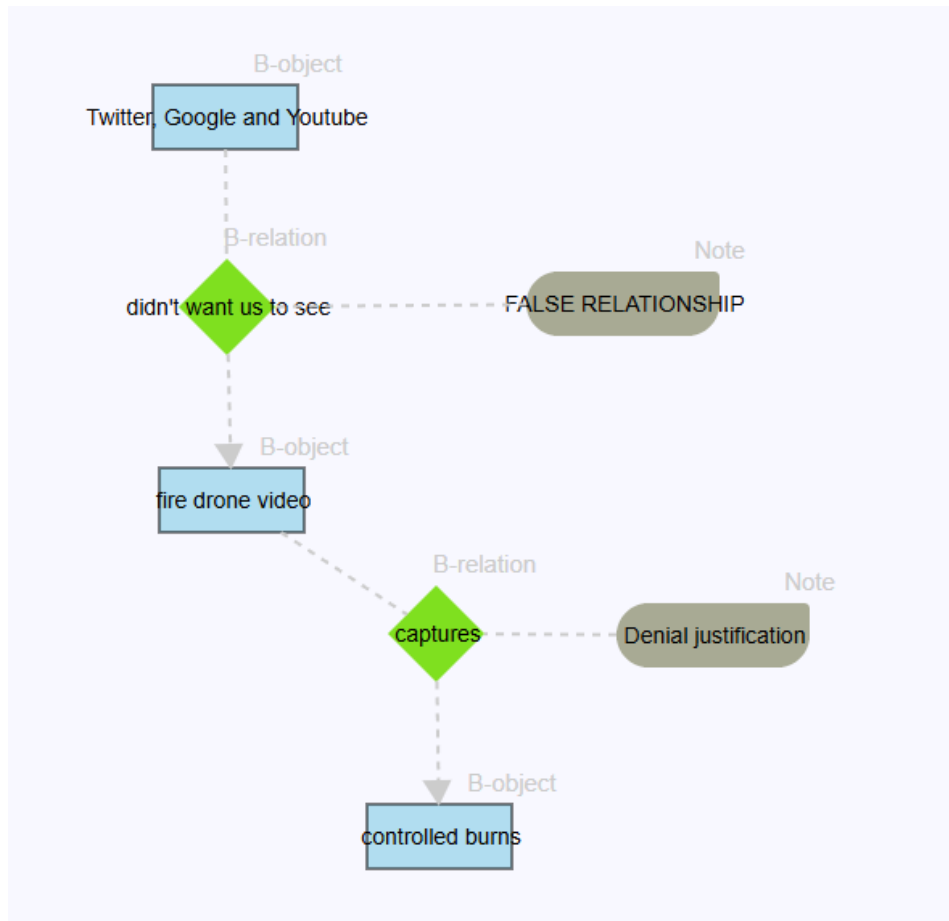
- The video shows a helicopter performing controlled burns, not a fire drone.
- Social media platforms never attempted to suppress or hide this video. The footage simply documents a routine forest management practice that requires no censorship.

This case demonstrates a common challenge in annotation: choosing between multiple valid argumentation elements. We could model the drone/helicopter misidentification as FALSE OBJECT (or perhaps FALSE TYPE, since both are flying objects), which would accurately capture one deceptive aspect of the claim.

However, the more concerning deception lies in the conspiracy narrative. The claim deliberately tries to convince viewers that social media platforms are hiding important information from them. This manufactured censorship story transforms routine footage into supposed evidence of a cover-up. Without this conspiracy angle, a simple misidentification of aircraft would likely generate little interest and therefore little harm.

Figure 1.2

The "fire drone" fact-check report entity graph



Source: Author

Therefore, I decided to model this as FALSE RELATIONSHIP between the platforms and the video (Figure 1.2). It is completely untrue that the authorities "didn't want us to see" this content. While the helicopter/drone confusion is worth noting, it doesn't drive the claim's potential for harm. By focusing on the primary deception, we create a cleaner model that captures what makes this claim truly problematic: its attempt to fuel distrust in social media platforms through a fabricated censorship narrative.

Additionally, the model also contains an auxiliary argumentation element - the *denial justification*, which cites the fire department spokesperson. This captures more information from the fact-check, making the visualization more explainable.

After analysing the examples, we can observe that misinformation patterns consist of two essential components: "what is wrong" in the claim and "how it is wrong" - essentially identifying both the problematic element and the nature of the misinformation. This thesis aims to investigate whether large language models can automatically detect these misinformation patterns and provide short, comprehensible explanations of what makes the claims problematic.

2 Machine Intelligence in Fact-Checking

The purpose of this chapter is to map the current state-of-the-art techniques of leveraging natural language processing for fact-checking. This review focuses specifically on fact-checking tasks, which encompass: (1) claim detection, (2) gathering relevant resources that support or refute claims, (3) verdict determination based on evidence, and (4) summarization of fact-checking findings.

2.1 Claim detection

Before we can begin analysing a claim and classify whether it is problematic, we need to identify it first. Detecting a misinformation at an early stage is a crucial step to stop it from spreading in the online space. With the huge amount of information being transmitted online, relying on manual checks by human labelers quickly becomes unsuitable. Therefore, automatic approaches need to be employed.

The study *Claim Detection for Automated Fact-checking: A Survey on Monolingual, Multilingual and Cross-Lingual Research* presents three main subtasks of claim detection: verifiability, priority, and similarity (Panchendrarajan & Zubiaga, 2024).

2.1.1 Verifiability

A claim is considered verifiable when it makes a factual assertion about the world that can be objectively checked against evidence (Panchendrarajan & Zubiaga, 2024). This definition is crucial because not all statements can be fact-checked. For example, subjective personal experiences cannot be verified with any external evidence. A statement such as "Today, I felt motivated to write my bachelor's thesis" represents a personal feeling that cannot be proven or disproven through factual investigation.

Konstantinovskiy et al. (2020) note that in 2015, the fact-checking organisation Full Fact proposed a concise definition of a claim as *an assertion about the world that can be checked*. However, this definition proved to be too broad for practical application, prompting the researchers to develop a more systematic approach to claim identification. They introduced a classification framework that categorizes sentences into seven distinct types. Trained annotators assigned each sentence to one of seven predefined categories, which were then mapped to binary labels: *claim* or *not a claim*. A sentence was qualified as a claim if it belonged to one of the following five categories:

- **Quantity in the past or present:** Including statistics, numerical comparisons, and rankings
- **Correlation or causation:** Statements linking two or more phenomena
- **Current laws or rules of operation:** References to procedures, regulations, or institutional practices

- **Predictions:** Hypothetical assertions about future events or outcomes
- **Other verifiable claims:** Such as voting records, public opinion data, statements of support, or definitions (Konstantinovskiy et al., 2020)

Sentences classified as *personal experience* or *not a claim* were excluded from fact-checking. This framework aims to make the definition practical: a claim is any statement that fits within one of the five verifiable categories.

2.1.2 Priority: Determining check-worthiness of claims

The second component of claim detection is figuring out which claims should be fact-checked. Fact-checkers can't possibly verify each detected claim manually; prioritization thus becomes necessary.

The most important factor is the potential for harm. Claims about public health emergencies, electoral manipulation, or violent events must be addressed promptly, as believing false information in these areas can have real-world consequences. For example, during the COVID-19 pandemic, false claims about vaccine ingredients or alternative cures posed direct threats to public health and were prioritized by fact-checking organizations worldwide.

The speed at which a claim spreads and its reach also matter. A false statement shared by a well-known figure or one that goes viral on social media requires quick intervention. Research shows that misinformation spreads six times faster than trustworthy news on platforms like Twitter (Vosoughi et al., 2018).

Check-worthiness also depends on practical factors. Some claims require extensive investigation by experts, while others can be quickly verified using public sources. Claims made during election campaigns or regarding pending legislation receive extra attention because of their timing and potential to impact democratic processes.

2.1.3 Similarity: Identifying repeated claims

The third component of claim detection involves recognizing when a new claim is a restatement or variation of a statement that has already been fact-checked. False claims rarely appear just once; they resurface in different forms, especially during critical events.

The claim matching process works by comparing new statements against databases of previously verified claims. When fact-checkers find a match, they can immediately apply the existing verdict instead of starting a fresh investigation (Panchendrarajan & Zubiaga, 2024).

The real challenge is that claims mutate. A statement like "wind turbines cause cancer" could return as "windmills produce harmful radiation" or "living near wind farms causes illness." The words change, but the core idea of the claim stays the same. Natural language processing (NLP) techniques help identify these variations by analysing meaning rather than just matching keywords.

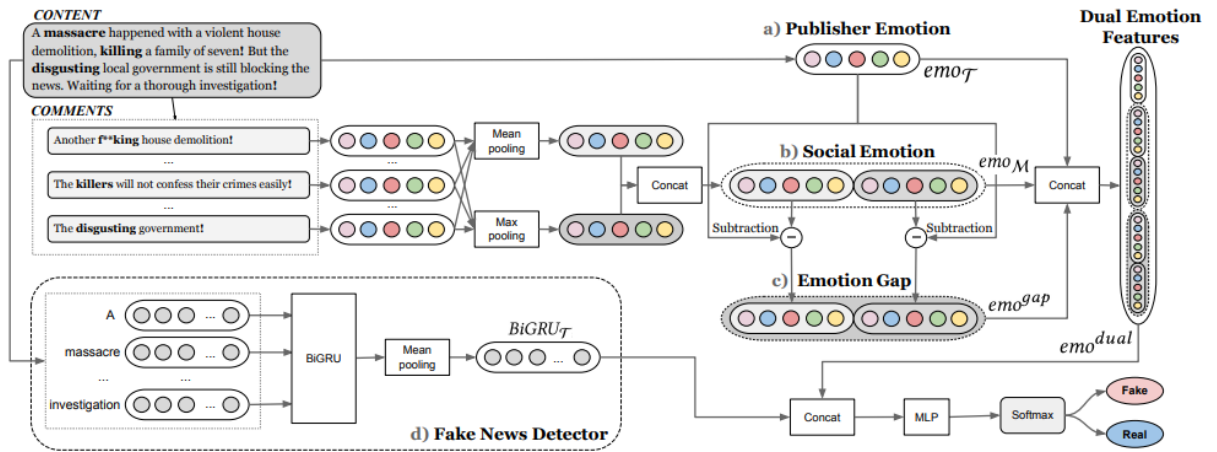
2.1.4 Automatic Claim Detection Approaches

Guo et al. (2022) states that automatic check-worthy claim detection is challenging since the volume of online posts published makes it impossible to do a retrieval of evidence for each of them. *Rumorous and non-rumorous information is often difficult to distinguish.* Therefore, the task has to be framed as a classification problem that works only with the limited information that is available. Such baseline methods are based on analysing only the actual post. For example, text analysis can be performed on the content and compared to identify similarities with other previously detected misinformation. These approaches work on the assumption that misinformation tends to use a different style of language, often employing more emotional language.

This approach is explored by X. Zhang et al. (2021), which examines the emotion of the original post (i.e., the publisher emotion) and the emotion present in reactions to the posts, typically the comments. This relationship is termed the *dual emotion*. The work proposes calculating a dual emotion feature set and demonstrates that it can enhance the performance of fake news detectors. The proposed features use an emotion classifier which outputs probabilities that certain emotions are present in a text and combines them with various other indicators such as the frequency of angry emoticons or exclamation marks. The overview of the system is presented in Figure 2.1.

Figure 2.1

The framework leveraging dual emotions for fake news detection



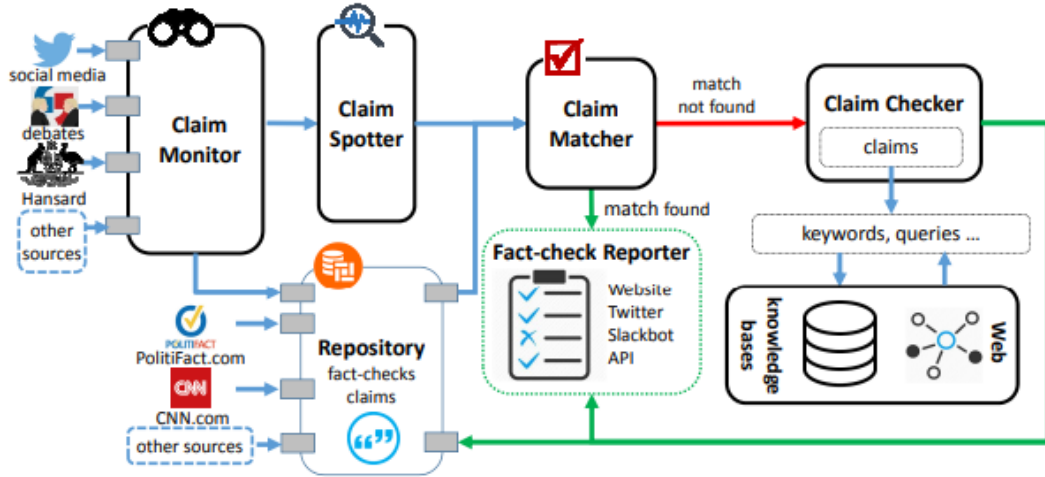
Source: (X. Zhang et al., 2021)

Hassan et al. (2017) developed ClaimBuster, a project that employs supervised learning to identify check-worthy factual claims (Figure 2.2). The authors created a platform where annotators labeled statements into three categories: *Non-Factual Sentence* (NFS), *Unimportant Factual Sentence* (UFS), and *Check-worthy Factual Sentence* (CFS). This annotated dataset was then used to train a classification model. The statements were represented using multiple features, including sentiment scores, sentence length, TF-IDF values, part-of-speech tags, and entity types. Using an SVM classifier, accuracy of 72% was

achieved in identifying CFS sentences. Beyond claim detection, the ClaimBuster project expanded to include a comprehensive platform featuring a database of verified factual statements.

Figure 2.2

The components of the ClaimBuster platform



Source: (Hassan et al., 2017)

The next section examines how machines can assist in gathering relevant resources and determining verdicts once a claim worth investigating has been identified.

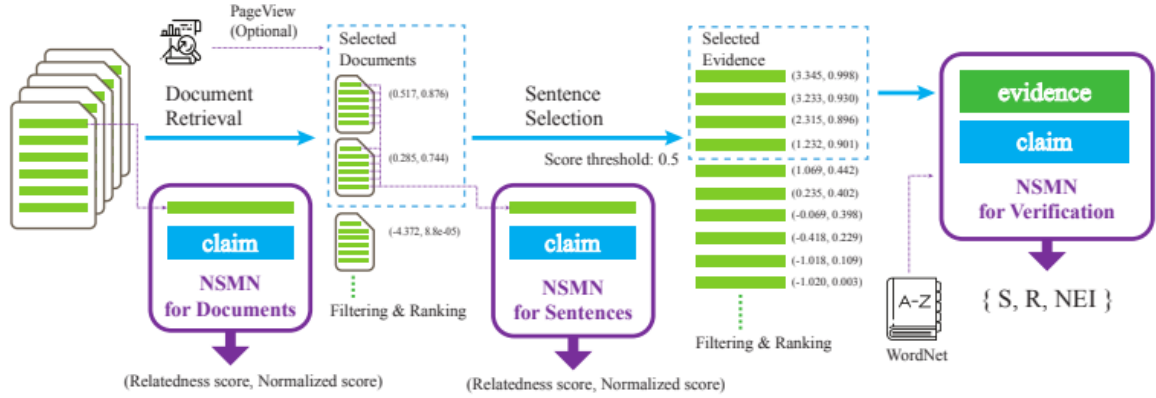
2.2 Gathering Relevant Resources & Verdict Determination

Nie et al. (2018) describe a solution that combines three models to retrieve and verify statements from the FEVER dataset (Fact Extraction and VERification). This dataset, created by Thorne et al. (2018), contains *185,445 claims generated by altering sentences extracted from Wikipedia*. The claims in the dataset are verified without knowledge of their

original source sentences, which creates a challenging benchmark for automated fact-checking systems.

Figure 2.3

Solution architecture for automatic document retrieval, sentence selection, and claim verification



Source: (Nie et al., 2018)

The solution divides the task into three stages, which are sketched in Figure 2.3: Document Retrieval (selecting relevant documents from Wikipedia), Sentence Selection (identifying relevant sentences from the retrieved documents), and Claim Verification (classifying claims into three categories: *SUPPORTS*, *REFUTES*, or *NOT ENOUGH INFO*). All three models employ a *Neural Semantic Matching Network* (NSMN) that takes two textual sequences as input. The output varies by task: for extraction subtasks (document retrieval and sentence selection), the model produces a two-dimensional vector containing scores for selecting or discarding evidence. For claim verification, it generates a three-dimensional vector with scores corresponding to each classification label. The system achieved F1 scores of 75.7, 69.4, and 63.3 for the *SUPPORTS*, *REFUTES*, and *NOT ENOUGH INFO* categories, respectively, on the FEVER dataset (Nie et al., 2018).

From the perspective of this thesis, the claim verification task is particularly relevant. Whilst Nie et al., (2018) employed supervised learning with LSTM-based models (discussed further in Section 2.3.2 *Long Short-Term Memory Unit*), I believe current LLMs would excel at this task. Large language models could process the evidence directly and reason about which category best fits the claim, potentially offering more nuanced verification capabilities.

2.3 Text Summarization

Text summarization is the task of creating a shorter representation of a text while keeping as much important information as possible. In the misinformation domain, summarization plays an important role in making fact-checking more accessible. Fact-check reports are often long and complicated as they thoroughly analyse claims and present evidence,

therefore they can be challenging for the general audience to digest. Automated summarization can help by making the reports shorter, which makes them more accessible for the readers. This is particularly important in combating misinformation, as the summaries are more likely to be read and shared, potentially reaching the same audience that encountered the original false claims.

We can divide text summarization into two main categories - extractive summarization and abstractive summarization.

2.3.1 Extractive summarization

Extractive summarization is carried out by selecting the most important parts of a text and keeping them in their original form. The usual granularity at which the text is selected is at the sentence level. The chosen sentences are put together to form a subset of the original text - the extractive summary. As we can see, the task is similar to claim detection because we are also selecting a subset of a text to keep. Nevertheless, there are some key differences. Claim detection can select heterogeneous parts of the text (each part being associated with a different claim) and does not necessarily aim to summarize the original text. On the other hand, extractive summarization chooses the most relevant parts of the text to represent it as accurately as possible as a whole.

Extractive summarization has been a subject of extensive research, especially before the rise of LLM-related methods, which enabled effective abstractive summarization. The earliest methods date back to the 1960s. In *The Automatic Creation of Literature Abstracts*, the authors introduced a fully automatic way of creating extractive summaries using statistical methods like word frequency and distribution (Allahtari et al., 2017; Luhn, 1958).

Text Selection

A key idea for the process of creating an extractive summary is to assign sentences with an *importance score*. Once all sentences are scored, we can either select the top-k most important sentences to form a summary or use a more sophisticated algorithm which can also optimize factors like coherency or take into account context (Allahtari et al., 2017).

The *importance score* is calculated based on an intermediate representation of the text. Historically, there were two main approaches. *Topic representation* aims to first detect the topics present in the whole text and then scores the sentences based on how relevant they are to the topics. On the other hand, the core idea of *indicator representation* is to *describe every sentence as a list of features (indicators) of importance such as sentence length, position in the document, having certain phrases, etc.* (Allahtari et al., 2017).

2.3.2 From Abstractive Summarization to Large Language Models

In contrast to the extractive approach, an abstractive summary can contain sentences and words that are not present in the original text. Naturally, it requires a more sophisticated approach. In the 2010s, deep learning based approaches have made a big leap forward in tasks like machine translation, voice recognition, and dialogue systems. Especially the

category of Sequence-to-Sequence models (Seq2Seq) proved to be well suitable for text summarization. They process text as a sequence of tokens, extract semantic information using recurrent hidden neurons, *and recursively generate sequential tokens as summaries. Examples include Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU)* (Y. Zhang et al., 2025).

Recurrent Neural Networks

Recurrent Neural Networks can often better capture patterns in a sequence of data compared to the more classic Feed Forward Neural Networks (for example, the *Multilayer Perceptron* - MLP). The data is not just passed in one direction through the layers but also in cycles. This enables RNNs to transmit information back to itself. Hence, previous inputs $\mathbf{X}_{0:t-1}$ can be taken into account. In contrast, MLPs work only with the current input $\mathbf{X}_t$. RNNs often struggle with the problem of vanishing or exploding gradients. The forward pass inside of the neural network is calculated using a sequence of matrix multiplications. If the matrix contains very small numbers (< 1), the gradient decreases with every step of the calculation and will eventually vanish. *Similarly, large values will cause the gradient to explode which will cause the neural network to value each weight too much which changes it heavily* (Schmidt, 2019).

Long Short-Term Memory Unit

Long Short-Term Memory Units and the Gated Recurrent Units are designed to handle the vanishing gradient problem better by having more constant errors. Together with RNNs, they can be used as layers in a seq2seq models, which consist of a more advanced encoder-decoder architecture. The encoder maps the input to a hidden state, which acts as an input for the decoder part of the model. For example, this architecture is used to power the Google Translator (Schmidt, 2019; Wu et al., 2016).

All architectures above struggle with the same task - working with long text sequences. RNNs have the vanishing gradient problem which gets amplified the longer the input text is. The Encoder-decoder architecture is limited by the fixed length of its encoder vector. It must contain all context from the input, which is not always possible, especially when working with long sentences.

The Attention Mechanism

Motivated by the need to solve the issue with fixed-size encoder vectors in seq2seq models, a novel technique has been introduced in late 2014 - the attention mechanism. The core idea is that the decoder does not necessarily need to work with the entire encoder vector. Instead, in each step during the process of generating output, it can pay attention to different parts of the entire input text. This enables the decoder to selectively take into account the words that are important for generating the next word. Hence, the model is able to work with much longer sequences without losing important context.

Combining attention with RNNs significantly improved the model's ability to understand text. In the paper that originally introduced attention, the authors benchmark it on a

translation task. *We tested the proposed model, called RNNsearch, on the task of English-to-French translation. The experiment revealed that the proposed RNNsearch outperforms the conventional encoder–decoder model (RNNencdec) significantly, regardless of the sentence length and that it is much more robust to the length of a source sentence* (Bahdanau et al., 2016).

The Era of Transformers

In 2017, arguably the most influential artificial intelligence paper in recent years was published. In *Attention is All You Need* by researchers from Google Brain, it has been proposed to replace the recurrent (or convolutional) layers with attention completely. The novel model architecture with multi-head attention layers has been named the *Transformer*. The carried-out experiments proved it to outperform the state-of-the-art approaches while being more efficient to train compared to RNNs (Vaswani et al., 2023).

In addition, the transformer architecture allows for great parallelization. In other words, the models can be scaled to a large number of layers and capture patterns from trillions of training tokens. As models grow larger, they begin to exhibit emergent capabilities such as an unprecedented ability to understand text, surpassing what previous approaches could achieve. These are what we refer to as the *large language models*.

Unlike conventional abstractive summarization methods that require task-specific training, LLMs can perform summarization out of the box (Y. Zhang et al., 2025). This eliminates the need for dedicated training datasets and procedures for each summarization task. However, LLMs present challenges such as not following instructions precisely or generating summaries that exceed specified length constraints. These issues are typically addressed through prompt engineering, few-shot examples, or fine-tuning approaches like instruction tuning or reinforcement learning from human feedback. The first two techniques are explored in the practical section of this work.

2.3.3 Summarization in Fact-Checking

How to effectively condense fact-check reports whilst preserving their essential information has been the subject of research at VŠE.

Vajdecka et al. (2023) propose a hybrid summarization approach specifically designed for fact-checking content, combining Local Outlier Factor (LOF), Sentence-BERT (SBERT), and T5 transformers. Their method first performs extractive summarization by using fine-tuned SBERT to create sentence embeddings, then applies LOF to identify and remove less relevant outlier sentences. The retained sentences form an extractive summary, which is subsequently fed into a fine-tuned T5 model to generate a concise abstractive summary. When evaluated on fact-checking datasets from PolitiFact and Demagog.cz (Czech), using human-written justifications as reference summaries, their approach achieved state-of-the-art ROUGE scores, outperforming baselines including GPT-2 and TextRank variants.

2.4 Summary and Outlook

In this chapter, we explored the theoretical foundations of automatic fact-checking. Based on the reviewed literature, we can observe that fact-checking tasks generally fall into two categories: claim detection (determining what needs verification) and claim verification (assessing truthfulness). Research typically focuses on individual components (or a combination of multiple components) rather than end-to-end systems. Claim detection remains challenging to automate, with content often being fact-checked only after gaining viral attention. For verification tasks, approaches like those by Nie et al. (2018) demonstrate how similarity metrics can retrieve relevant information from knowledge bases, followed by supervised learning for claim classification into verdict categories. Our examination of summarization traced natural language processing evolution to large language models, which offer unprecedented versatility for fact-checking applications, though they present challenges in instruction following and output control.

The experimental part of this thesis builds upon the discussed approaches. Unlike traditional claim verification, which evaluates standalone claims, the task being solved in this thesis analyzes claims that have already been fact-checked. While this approach currently requires a human-written fact-check report as an input, its significance extends beyond this limitation. For example, as automated fact-checking systems improve, this method could help explain and evaluate their outputs by analyzing their argumentation and labeling it with the detected core misinformation patterns.

3 Annotation Campaign

This thesis contributes to the *Climate Sense* project, an international research initiative led by Prof. Harith Alani from the Open University (UK), with partners from EURECOM (France), Kaunas University of Technology (Lithuania), and Prague University of Economics and Business (Czech Republic). The Climate Sense project aims to understand how climate-related misinformation spreads across different platforms and languages, and to develop effective methods for detecting and countering such misinformation through automated analysis.

To evaluate the performance of large language models in detecting misinformation patterns, it was first needed to establish a reliable ground truth dataset through human annotation. This chapter describes the systematic approach we used to gather and annotate climate-related fact-checking reports, creating the foundation for the subsequent experiments with LLMs.

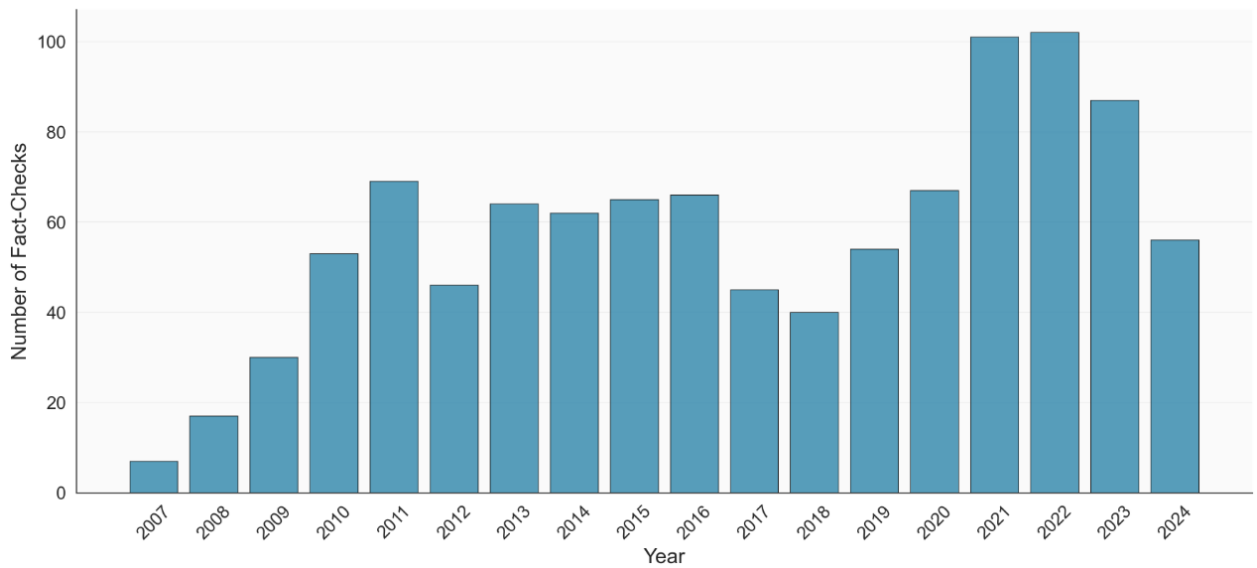
Our annotation campaign builds upon previous work by the research team from VŠE, specifically their paper *Towards Fact-Check Summarization Leveraging on Argumentation Elements Tied to Entity Graphs* (Haniková et al., 2024). While their research focused on effective summarization of fact-checks, this thesis extends the methodology to automated detection and explanation of the misinformation patterns present within the climate domain.

We selected PolitiFact as our primary source, scraping over 1,000 fact-check reports labeled with *climate* as a topic.⁸ Each report in the dataset contains several key components: the original claim being checked, the fact-checker's detailed reasoning, an *If Your Time Is Short* summary section, and the final verdict (ranging from *True* to *Pants on Fire*). These structured reports provide rich material for understanding how fact-checkers identify and explain various forms of climate misinformation.

⁸ The dataset is available at https://github.com/Rigos0/fact-check-modeling/blob/main/data/politifact_climate_dataset.xlsx

Figure 3.1

Fact-check publication frequency on PolitiFact

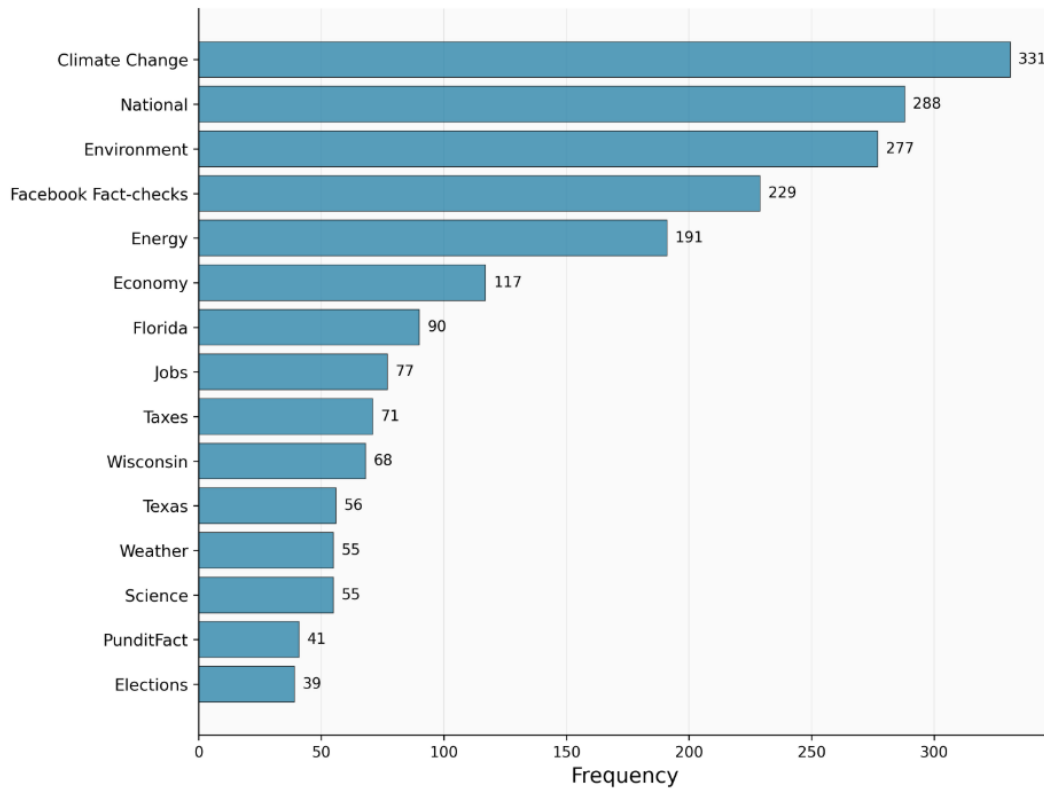


Source: Own processing based on data scraped from PolitiFact

Figure 3.1 shows the temporal distribution of the fact-check reports scraped from PolitiFact. There is a significant spike in 2021 and 2022, which is possibly caused by the increased dissemination of misinformation during the COVID-19 crisis.

Figure 3.2

Most frequent topic labels in climate related fact-checks from PolitiFact



Source: Own processing based on data scraped from PolitiFact

Figure 3.2 displays how often different topic labels appear alongside *Climate* in the dataset. The prevalence of *National* (288) and *Environment* (277) tags suggests that climate misinformation frequently appears in discussions of national policy and environmental issues.

3.1 Methodology of Annotation

We selected 50 fact-check reports from the past five years, focusing on recent content to ensure relevance to current climate discourse and misinformation trends.

Our annotation team consisted of five researchers; each was assigned 20 fact-check reports to analyse. The distribution was designed so that each annotator shared exactly 10 fact-checks with another colleague. Hence, every report received annotations from two independent annotators, which allowed us to assess consistency and capture different perspectives on the misinformation patterns present in each text.

Prior to beginning the annotation process, all team members were trained on the *Catalogue of Misinformation Patterns*.⁹ This catalog provides a comprehensive guide for analysing claims by modeling them using entity graphs. It defines common argumentation elements and misinformation patterns. It also contains multiple annotated examples of how to model the argumentation in fact-checked reports.

The annotation task consisted of three main components. First, annotators categorized the misinformation pattern by identifying which specific pattern from the catalog best described the deceptive reasoning in the claim. In the second step, they selected key snippets from the full fact-check report, extracting the most important text passages that supported their decision. Finally, annotators created a short explanation describing why the identified pattern applied to the specific claim.

Following the initial independent annotation phase, we conducted reconciliation meetings where pairs of annotators who had examined the same fact-check reports discussed their findings. During these sessions, annotators compared their identified argumentation elements, resolved discrepancies, and either reached consensus on a single verdict or agreed that multiple interpretations were equally valid. The reconciliation process improves the quality and reliability of the final dataset and provided valuable insights to all members of the team.

3.2 Annotation Campaign Results

In the annotation campaign, each annotator identified the most prominent verdict argumentation element for their assigned fact-check report. In this section, I analyse the level of agreement between annotators to understand the consistency of the framework and the clarity of different argumentation patterns.

We measured inter-annotator agreement at two stages: after the initial independent annotation phase and following the reconciliation meetings. Initial agreement levels were categorized into three classes:

- **Perfect Match:** Both annotators selected identical verdict argumentation elements.
- **Partial Match:** Annotators chose verdict argumentation elements that match either in the adjective or the noun.
- **Different Decision:** Annotators selected different verdict argumentation elements.

The initial agreement rates, before any discussion between annotators, were as follows:

⁹ Available at https://github.com/Rigos0/fact-check-modelling/blob/main/catalogue_of_argumentation_patterns.pdf

Table 3.1*Annotation campaign - initial agreement rates*

Initial Agreement Level	Count	Percentage
Perfect Match	19	38%
Partial Match	19	38%
Different Decision	12	24%

Following the reconciliation meetings, the annotators either agreed on a single verdict argumentation element or acknowledged that multiple elements were equally applicable.

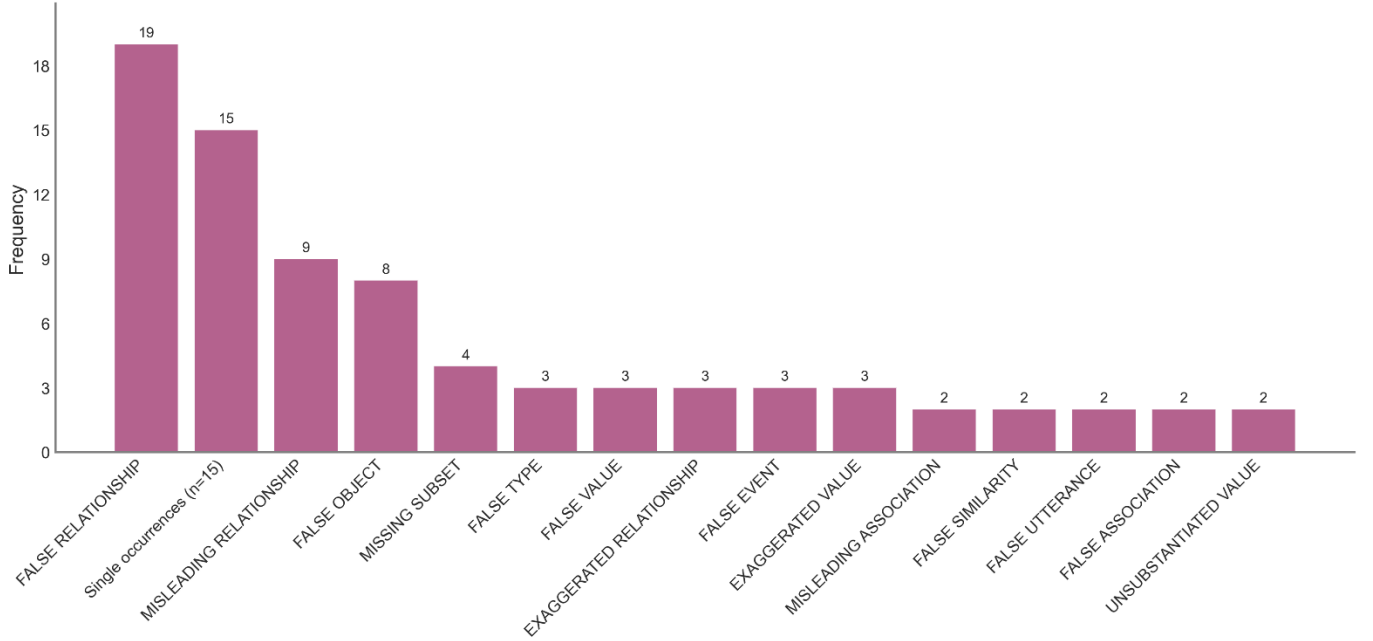
Table 3.2*Annotation campaign - agreement rates after reconciliation meetings*

Agreement After Meeting	Count	Percentage
One Solution	27	54%
Multiple Solutions Possible	23	46%

The reconciliation process improved consensus, with over half of the cases converging to a single solution. A substantial portion of cases (46%) where multiple solutions remained valid demonstrates the inherent difficulty in annotating certain fact-checks and shows that different interpretations of the same content can be equally valid.

Figure 3.3

Distribution of misinformation patterns selected in the annotation campaign



Note: The category *single occurrences* encompasses all misinformation patterns that have been selected only once

Source: Author

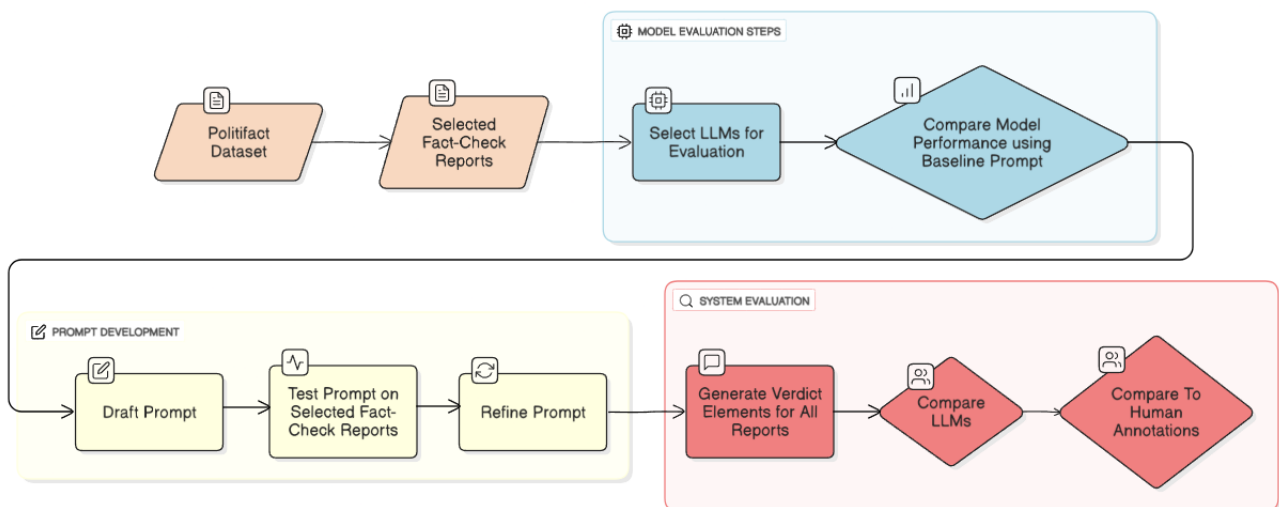
Figure 3.3 presents the distribution of verdict argumentation elements following the reconciliation process between annotators. This manually annotated distribution serves as our ground truth baseline against which we will evaluate the automatically generated results in the main experiment of this thesis.

4 Automatic Classification of Misinformation Patterns

This chapter describes the first practical part of this thesis, specifically tackling RQ1. The goal is to develop a system that takes a fact-check report as input and outputs the corresponding verdict argumentation element (misinformation pattern). The experiments begin by comparing different LLMs with the goal of identifying the most suitable candidates for this task. Additionally, how consistently the models select the same misinformation pattern when queried with the same prompt multiple times is evaluated. Afterward, different prompts are iteratively tested and refined to optimize model performance. The approach is evaluated by generating verdict argumentation elements for all fact-check reports from the PolitiFact dataset used in the annotation campaign. These outputs are then compared across different models and benchmarked against the classifications decided by human annotators.

Figure 4.1

Overview of the experiment workflow



Source: Author

4.1 Selecting Fact-check Reports for Development

For the purpose of initial model evaluation and prompt development, a small set of representative fact-check reports was selected. The selection includes the two reports previously analysed in Chapter 1.2.1, which enables detailed comparison with the annotators' modeling. The first report "document signing utterance" is very simple and serves as a straightforward test case. The second report about the fire drone video proved

to be a great example to demonstrate how LLMs can fall into simple pitfalls - focusing on minor misinformation that, while being factually incorrect, does not represent the key problem addressed in the fact-check report.

A third report was selected for its length and complexity to test the models' ability to handle more challenging content. This fact-check examines the claim that states: *On key issues including taxes, health care, the Green New Deal, abortion, and guns, there is 'no daylight between' Bernie Sanders and Joe Biden.*¹⁰ The fact-check report addresses each issue mentioned, making the argumentation significantly longer compared to other examples. In the annotation campaign, two potential verdict elements were found: *EXAGGERATED SIMILARITY* and *MISLEADING RELATIONSHIP*.

Table 4.1

Overview of the fact-check reports selected for development

Fact-check Report	Number of Words	PolitiFact Verdict	Annotation Verdict Element	Campaign
Biden Utterance	568	False	FALSE UTTERANCE	
Fire Drone Video	367	False	FALSE RELATIONSHIP (+ FALSE OBJECT)	
No daylight between Sanders and Biden	1,655	Half True	EXAGGERATED SIMILARITY MISLEADING RELATIONSHIP	and

In the following sections, the fact-check reports described in Table 4.1 are used to benchmark different approaches and LLMs.

4.2 Selecting Suitable Large Language Models

New LLMs are released at a considerably high frequency, typically accompanied by evaluations on standardized benchmarks. However, companies developing these models are incentivized to highlight benchmarks where their models excel, because of the competitive nature of the AI field. There have also been instances of models being specifically optimized for certain benchmark types. Such models might demonstrate exceptional performance on medium-difficulty programming problems, for example, yet fail to generalize to other, more challenging tasks. This phenomenon represents a modern

¹⁰ Full report available at: <https://www.politifact.com/factchecks/2020/mar/10/republican-national-committee-republican/there-no-daylight-between-joe-biden-and-bernie-san/>

manifestation of overfitting, though the term is more traditionally used in more classical machine learning (ML) approaches.

One of the most popular evaluation methods started as a small hackathon project. This website, known as *ChatBot Arena* or *LMarena*¹¹, allows users to submit the same prompt to two anonymous models. Users then evaluate which model provided a better response and cast their vote. The votes are aggregated and used to calculate scores based on the ELO rating system (Chiang et al., 2024).

Nevertheless, even LMarena has been subject to recent controversy. The company Meta submitted models specifically optimized to appeal to LMarena voters, whilst releasing a different set of models open-source (Singh et al., 2025). This manipulation was possible because previous "model battle" votes were publicly available, enabling training on this dataset. Consequently, the Llama-4 models temporarily achieved disproportionately high rankings on the leaderboard relative to their actual capabilities.

The reason for discussing these evaluation challenges is to illustrate that determining which model is best suited for a specific task is not straightforward. Therefore, this thesis aims to test a broader range of models to ensure a comprehensive and robust evaluation.

Six models were selected for evaluation - two from each of the three major providers: OpenAI, Anthropic, and Google. The selection includes both the most advanced model and a standard model from each provider. For instance, while OpenAI is primarily known for their ChatGPT-4o model, which is freely accessible through the web interface, their most capable offering is the o3 model, which is a *reasoning model*.¹²

Additionally, I selected the highest-ranked open-weights model, which is currently the Chinese reasoning model *DeepSeek R1 0528*. I believe open-source models can play a crucial role in fact-checking applications, as end users can run them on their own hardware and maintain certainty about which model and configuration is providing responses. This transparency is especially important since models accessed through APIs can be modified at any point, potentially altering their stance on sensitive topics without notifying the end users. While open-source models do not eliminate issues such as opinion biases due to the black-box nature of LLMs, they at least guarantee that responses consistently come from the exact same model version and configuration when deployed privately.

¹¹ <https://lmarena.ai/>

¹² Reasoning models represent a recent advancement in LLM training. These models are specifically trained to automatically "reason" through problems using semi-hidden reasoning tokens before generating their final output. This approach has demonstrated significant improvements in model performance, particularly for complex tasks requiring problem-solving abilities.

Table 4.2*Comparison of large language models selected for initial evaluation*

Model Name	Provider	Reasoning Model	License	LmArena Score *	LmArena Overall Rank (UB) *	Price per 1M Output Tokens (\$)
ChatGPT-4o	OpenAI	No	Proprietary	1439	3.	\$15
OpenAI: o3	OpenAI	Yes	Proprietary	1449	2.=	\$8
Claude Sonnet 4	Anthropic	No	Proprietary	1394	11.=	\$15
Claude Opus 4	Anthropic	No	Proprietary	1410	6.=	\$75
Gemini 2.5 Flash Preview 05-20	Google	No	Proprietary	1418	6.=	\$0.60
Gemini 2.5 Pro Preview 06-05	Google	Yes	Proprietary	1468	1.	\$10 - 15
DeepSeek: R1 0528	DeepSeek	Yes	MIT	1410	6.=	\$2.15

*Source: Own processing based on data from lmarena.ai**Note: *The scores reflect ranking as of 17.6.2025*

I also tested the latest model from xAI, labeled *Grok 3 Beta*, and the highest ranked model trained by a European company - *Mistral-Medium 2505*. However, I ultimately decided to exclude them from the statistical analysis, as their performance did not demonstrate any particular advantages that could make them optimal choices for the verdict element determination task.

4.3 Evaluating LLM Consistency

Before evaluating how well different models perform, it was crucial to assess consistency across responses. An experiment was conducted using the selected fact-check reports, where each model was prompted five times with identical input.

The test was run using a baseline prompt:

Prompt 4.1

Verdict argumentation element determination - baseline prompt template

```
Using the argumentation catalog below, identify the single most prominent verdict
argumentation element in the provided fact-check report. Keep your response brief, following
this format:
"""
short explanation: <explanation>
element: <element>
"""
Catalog: <part of the argumentation catalog (overview and patterns explanations)>
Fact-check report: <full text of the Politifact fact-check report>
```

Model API documentation was followed to maximize reproducibility. For most models, this involved setting the temperature parameter to 0, which reduces randomness by making the model select the most probable next token. The OpenAI API includes an experimental seed parameter designed to enhance determinism in responses. However, the documentation acknowledges that complete determinism cannot be guaranteed due to model configuration changes on the server backend (*Advanced Usage - OpenAI API*, n.d.).

Notably, LLM determinism is not guaranteed even when the configuration is kept exactly the same. *While temperature indeed affects determinism, other model-specific factors, such as floating-point precision, token sampling quirks, architecture, and training data, also play significant roles. Moreover, some closed-source LLMs may deliberately limit determinism to safeguard proprietary techniques from reverse engineering* (Klishevich et al., 2025).

The consistency score metric is defined as

$$\text{Consistency Score} = \max(\text{count of each unique response}) / 5 \times 100\% \quad (4.1)$$

This metric measures the proportion of the most frequent response among the five trials, indicating the model's tendency to produce consistent outputs for identical inputs.

Table 4.3*Model consistency – "document signing utterance" fact-check report*

Model Name	Consistency Score	Valid Format*	Mode Verdict
ChatGPT-4o	100%	100%	FALSE UTTERANCE
OpenAI: o3	100%	100%	FALSE UTTERANCE
Claude Sonnet 4	100%	100%	FALSE UTTERANCE
Claude Opus 4	100%	100%	FALSE UTTERANCE
Gemini 2.5 Flash Preview 05-20	100%	100%	FALSE UTTERANCE
Gemini 2.5 Pro Preview 06-05	100%	100%	FALSE UTTERANCE
DeepSeek: R1 0528	100%	100%	FALSE UTTERANCE

Note: *Valid Format is defined as the percentage of solutions that contain a valid verdict argumentation element as defined in the Catalogue of Misinformation Patterns.

Interestingly, the initial experiment (Table 4.3) on the fact-check report that we annotators considered the simplest proved to be equally straightforward for all LLMs, achieving both full consistency across all five responses and complete agreement with human annotators across all models tested.

Table 4.4*Model consistency - "fire drone video" fact-check report*

Model Name	Consistency Score	Valid Format	Mode Verdict
ChatGPT-4o	100%	100%	FALSE OBJECT
OpenAI: o3	60%	100%	FALSE TYPE
Claude Sonnet 4	100%	100%	MISLEADING RELATIONSHIP
Claude Opus 4	100%	100%	MISLEADING ASSOCIATION
Gemini 2.5 Flash Preview 05-20	100%	0%	FALSE OBJECT, FALSE EVENT

Gemini 2.5 Pro Preview 06-05	100%	100%	MISLEADING RELATIONSHIP
DeepSeek: R1 0528	80%	100%	FALSE OBJECT

Even though the *Fire drone* fact-check report is shorter than the first example in terms of words, it proved significantly more challenging for the models (Table 4.4). Notably, two reasoning models failed to consistently provide the same verdict argumentation element, and Gemini 2.5 Flash failed to comply with the instructions when it provided two verdict argumentation elements instead of one.

Most importantly, the models selected five different misinformation patterns, which is a key observation for the development of our solution. It hints at the possibility of using multiple models to vote on the final verdict. Such an approach would be inspired by a solution known from machine learning as a *voting classifier*, which ensembles multiple weaker classifiers to improve the quality of the final decision.

The models' failure to reach consensus on a single misinformation pattern likely stems from different approaches to analysing the problem. This could be addressed by designing a better prompt that provides detailed steps for reasoning through the task and breaking it down systematically, which could theoretically improve model consensus. This approach is explored in Section 4.5 Prompt Refinement.

Table 4.5

Model consistency - "No daylight between Sanders and Biden" fact-check report

Model Name	Consistency Score	Valid Format	Mode Verdict
ChatGPT-4o	100%	100%	MISLEADING RELATIONSHIP
OpenAI: o3	80%	100%	MISLEADING IDENTITY
Claude Sonnet 4	100%	100%	MISLEADING RELATIONSHIP
Claude Opus 4	100%	100%	MISLEADING SIMILARITY
Gemini 2.5 Flash Preview 05-20	100%	0%	failed to provide full verdict element
Gemini 2.5 Pro Preview 06-05	100%	100%	MISLEADING SIMILARITY
DeepSeek: R1 0528	60%	100%	MISLEADING RELATIONSHIP

The third experiment, which covers the longest report, confirmed the consistency issues of the OpenAI o3 and DeepSeek R1 models (Table 4.5). This time, Gemini 2.5 Flash showed even greater non-compliance with the instructions, failing to provide the adjective of the verdict element. This is likely due to the model's degraded performance when handling longer contexts.

Notably, even though this report is five times longer than our second example, the models showed higher consensus. This confirms the idea discussed when annotating the examples in Chapter 1.2.1 that some fact-check reports are inherently more ambiguous or difficult to analyse than others.

The findings from this initial experiment phase prompted me to consider what makes a fact-check report complex versus simple. If we could automatically measure report complexity, it would provide valuable insight into the confidence level of our automated verdict element selection. Report complexity could also correlate with the PolitiFact verdict. I hypothesize that claims classified as "half true" are inherently more complex to explain compared to claims labeled as "false," as they require more nuanced argumentation.

Based on these considerations, I present three factors that could serve as parameters for measuring fact-check report complexity in the following section.

4.4 Fact-Check Report Complexity

The following factors are based on observations of which types of fact-check reports the LLMs struggled with the most during the experiments.

- **Entity density:** Each additional entity in a claim introduces new relationships and potential points of deception. A claim involving multiple interconnected elements naturally requires more complex modeling than one focusing on a single statement or relationship.
- **Contextual requirements:** The depth of background information needed varies dramatically between claims. Our first example required only expert testimony about audio clarity. In contrast, some claims demand extensive source documentation and expert knowledge across multiple domains. The more context required for verification, the more carefully we must balance completeness with clarity in our model.
- **Clarity of the verdict:** Claims rated as definitively false tend to be more straightforward to model than those labeled as half true or mostly false.

Some argumentation in fact-check reports can be modeled in multiple ways. The more complex the reasoning, the more possible representations emerge. However, effective modeling is not about capturing every detail but rather identifying the core deception. It is also important to note that the proposed modeling must follow the rules of the underlying modeling language, as described in the Catalogue of Misinformation Patterns.

4.5 Prompt Refinement

Although the initial experiment using the baseline prompt yielded a promising level of consistency across responses, Table 4.4 shows that the LLM's solutions differ from the solutions of human annotators. The following section describes how the prompt was iteratively improved to improve the LLM's reasoning over the underlying argumentation.

The task of improving the prompt can be compared to classical supervised learning. We have a dataset of ground truth - in our case, the annotated fact-checks from PolitiFact with their corresponding labels - the misinformation patterns found by the annotators. We need to train the model to learn how to achieve similar reasoning. However, in the case of prompt engineering, the problem is not about changing the internal parameters of the model but rather finding a prompt that produces the best results.

For the prompt development, I used the three examples we had already analysed in depth, along with several additional cases, iteratively improving the prompt until the models began identifying similar misinformation patterns to those selected in the annotation campaign. For instance, in the "fire drone" example, the models frequently selected FALSE OBJECT. However, as previously discussed, this is not the core issue in the argumentation. To understand their reasoning, I prompted the models to explain their decision-making process and adjusted the prompt accordingly. To address the issue of focusing on less important parts of the report, I added a section to the prompt starting with the text: *Focus on the most consequential or problematic sections in the claim - the one that shapes the audience's understanding in a deceptive or incorrect way...*

Throughout this iterative process, I consistently evaluated each prompt revision on the selected subset of reports to ensure that fixing incorrect behavior on one claim did not break the correct solutions we had already achieved in previous steps.

Key takeaways from this process included:

- The models showed a bias toward choosing high-level patterns, frequently defaulting to MISLEADING RELATIONSHIP. While this pattern can technically describe many fact-checks, it is crucial to provide more granular explanations when possible. Therefore, I added a section to the prompt stating: *Prioritize specificity: Choose the most specific and granular element...*
- Asking the models to directly provide the misinformation pattern yielded significantly worse decisions than allowing them to first reason through the problem. Therefore, I provided a template of steps for the model to follow before providing its final answer, beginning with: *Follow these steps: Write down the claim. Identify the key entities (objects, agents, events, etc.).*
- Providing extensive context and clear expectations proved to be essential. The argumentation catalog explains the task well for human annotators, but it was not specific enough for LLMs. To address this, I asked the models to write their own prompt based on the catalog. A significant performance improvement was achieved by simplifying the explanations of available misinformation patterns into a single list with concise descriptions.

- Even with these improvements, the models tended to approach the task using methods likely learned from their training data. To counter this, I included one complete annotated example with a breakdown of all entities, relationships, attributes, and values identified by the human annotator.

After combining these steps, we constructed a prompt that is significantly more detailed and explanatory than the baseline prompt. Notably, the prompt was developed in collaboration with the models themselves. When a model provided incorrect reasoning, I used prompts such as: *The provided argumentation is wrong because of... Explain why you solved the problem differently and suggest an improvement to the original prompt* to let the LLM suggest improvements for the next iteration.

Prompt 4.2 presents the complete prompt template developed through this process:

Prompt 4.2

Verdict argumentation element selection - refined prompt

Using the argumentation catalog below, identify the verdict argumentation element. The aim is to make the fact-check report explainable by modeling what is wrong with the original claim. The claim is always at the start of the fact-check report. You should choose a single verdict argumentation element – the core concept that explains why the claim is problematic.

Follow these steps:

Write down the claim.
 Identify the key entities (objects, agents, events, etc.).
 Identify the relationships between the entities (e.g., who did what, what is being claimed about whom or what).
 Ask: What is the claim trying to make the audience believe? What is the main message or implication?
 Decide which relationship is the most problematic – the one that drives the false or misleading narrative – and label it with the appropriate verdict argumentation element.

Important:

Focus on the most consequential or problematic sections in the claim – the one that shapes the audience's understanding in a deceptive or incorrect way.
 Don't just look for surface-level factual errors (e.g., wrong object type); instead, consider the broader implication or narrative the claim is promoting.
 If multiple errors are present, choose the one that best explains why the claim is problematic overall.
 Consider implied connections, not just explicitly stated ones. For example, a claim may imply a cause, connection, or suppression without directly stating it.

Catalog of Verdict Argumentation Elements:

A verdict argumentation element refers to what is wrong (or right) in the claim and in what way. It is composed of:

A noun that identifies what is wrong:

RELATIONSHIP – a specific factual relationship between two objects (including purpose, cause, context, or connection)
 IDENTITY – two objects are claimed to be identical
 SIMILARITY – two objects are presented as very similar
 ASSOCIATION – one object is linked to another to evoke a misleading connection
 CONTRAST – two objects are presented as opposites or very different
 ATTRIBUTE – an object is claimed to have a property it does not have
 VALUE – a quantitative value is incorrect or misleading
 OBJECT – the object itself is incorrect
 TYPE – the object is misclassified (e.g., a car is called a watch)
 SUBSET – a group is presented without acknowledging other relevant subgroups
 EVENT – something is claimed to have happened (or will happen) but did not occur at all
 UTTERANCE – someone is claimed to have said something they did not

An adjective that specifies how it is wrong:
 MISLEADING – technically true but presented in a way that leads to a false impression
 FALSE – factually incorrect
 UNSUBSTANTIATED – no evidence supports the claim
 MISSING – something important is omitted
 EXAGGERATED – the claim overstates the truth Provided in order "adjective noun"

Important Guidelines:
 Provide verdict using these brackets and format: <ADJECTIVE NOUN>
 Prioritize specificity: Choose the most specific and granular element that accurately captures the error. Avoid defaulting to broad categories like RELATIONSHIP, ASSOCIATION, or MISLEADING unless no more specific option applies.

Except of B-object, B-relation and verdicts, you may also use B-attribute (of an object) and B-value (quantitative of a attribute or object)

Example:

Claim: West Virginia University has “increased the amount of unfunded institutional aid provided to students and expects this amount to exceed
 We try to read the summary (the box “if your time is short”) to get an idea of what is wrong with the claim. It says that internally funded aid is poised to drop slightly in 2024 to about \$134 million. However, in the years before, there was growth. The claim causes the feeling that in 2024, it will rise as well. The core problem is highlighted in the claim above. So, we have detected two relationships (has increased and expects to exceed). These two relationships evoke some association – that in 2024, the unfunded aid will increase as it was the years before. Now, we can say that this association is MISLEADING ASSOCIATION. This is misleading because it is not false (the amount is correct, and the increasing trend before 2024 is also true).

Example solved:
 B-Entities:
 West Virginia University (WVU)
 Unfunded institutional aid
 Timeframe 2014 to 2023
 Timeframe 2024

B-Relationship 1: spent
 This relationship is derived from the part of the claim: "West Virginia University has increased the amount of unfunded institutional aid..."
 Subject: WVU
 On what: Unfunded institutional aid
 When: From 2014 to 2023
 B-Attributes:
 Trend -> B-Value: Increasing

B-Relationship 2: expects to spend
 This relationship is derived from the part of the claim: "...and expects this amount to exceed \$134 million in 2024."
 Subject: WVU
 On what: Unfunded institutional aid
 When: In 2024
 B-Attributes:
 Amount -> B-Value: More than \$134 million
 Trend -> B-Value: Increasing (This is implied by the association with the first relationship)

Verdict argumentation element: <MISLEADING ASSOCIATION> connecting b-relationships spent and expects to spend.
 Fact check to solve: {full fact-check text}

The refined prompt was evaluated using the same experimental setup described in Chapter 4.3 Evaluating LLM Consistency. For the simplest claim regarding the document signing utterance, the results were identical to those in Table 4.3, with all models achieving 100% accuracy.

For the second example, which is analysed in Table 4.6, we observed substantial improvement compared to the baseline prompt (Table 4.4). Nearly all models selected FALSE RELATIONSHIP, matching the annotator's decision. Notably, Claude Sonnet and Gemini 2.5 Pro achieved 100% consistency in selecting the correct solution.

Table 4.6

Refined prompt evaluation - "Fire drone video" fact-check report

Model Name	Consistency Score	Valid Format	Mode Verdict
ChatGPT-4o	40%	100%	FALSE RELATIONSHIP / FALSE EVENT
OpenAI: o3	40%	100%	FALSE RELATIONSHIP / FALSE EVENT
Claude Sonnet 4	100%	100%	FALSE RELATIONSHIP
Claude Opus 4	60%	100%	FALSE IDENTITY
Gemini 2.5 Pro Preview 06-05	100%	100%	FALSE RELATIONSHIP
DeepSeek: R1 0528	60%	100%	FALSE RELATIONSHIP

A clear improvement has been achieved even for the third example, which is analysed in Table 4.7 (compared to baseline prompt analysis in Table 4.5). In this example, both EXAGGERATED SIMILARITY and FALSE IDENTITY are valid solutions. Once again, the best performance was achieved by Gemini 2.5 Pro.

Table 4.7

Refined prompt evaluation - "No daylight between Sanders and Biden" fact-check report

Model Name	Consistency Score	Valid Format	Mode Verdict
ChatGPT-4o	60%	100%	FALSE IDENTITY
OpenAI: o3	40%	100%	EXAGGERATED IDENTITY
Claude Sonnet 4	60%	100%	FALSE IDENTITY

Claude Opus 4	60%	100%	EXAGGERATED SIMILARITY
Gemini 2.5 Pro Preview 06-05	80%	100%	EXAGGERATED SIMILARITY
DeepSeek: R1 0528	80%	100%	FALSE IDENTITY

The refined prompt shows promising results in selecting better misinformation patterns compared to the baseline prompt. However, a drop in consistency was also observed. This is because the model is asked to reason first before deciding on the answer, which creates higher variability in the text that is processed before selecting the misinformation pattern. I also tested a refined prompt that requested the solution to be provided at the top of the model's response. This approach, however, produced solutions similar to the ones generated using the baseline prompt - worse solutions, but high consistency across replies.

An interesting observation during the initial testing was that models trained by OpenAI tended to provide very similar solutions to each other, as did models from Anthropic. My initial intuition was that the largest models across companies (e.g., *Claude Opus* and *OpenAI o3*) would provide the most similar results, and that smaller models would more likely diverge. Instead, the pattern showed that models clustered by company rather than by size. This suggests that models from the same company are trained in similar ways, even though they have different numbers of parameters.

After the prompt refinement phase, I decided to continue the main tests with three models: Gemini 2.5 Pro as the overall best performing model, Claude Sonnet 4 as the best non-reasoning smaller model, and DeepSeek R1 since it also performs well and is open-weights. Notably, the OpenAI models consistently performed worse than the alternatives.

4.6 Main Experiment

The following section describes the main experiment of the thesis, in which we aim to detect misinformation patterns in 50 different fact-check reports automatically. In the previous chapter, the annotation campaign was discussed, during which the evaluation dataset was constructed. To reiterate, Table 3.2 presents that 27 claims have been labeled with a single possible solution, while the remaining 23 had multiple possible misinformation patterns selected by the annotators.

4.6.1 Experiment Overview

The experiment applies the prompt template from Chapter 4.5 to each of the 50 fact-check reports in the evaluation dataset. The selected misinformation pattern is then parsed out of the LLM's response automatically (in Prompt 4.2, we can see the model is asked to provide the solution consisting of two words in angle brackets).

Code Snippet 4.1

Parsing the misinformation pattern from the LLM's response using a regular expression

```
import re
from typing import List, Optional

def extract_bracketed_text(input_string: str) -> Optional[List[str]]:
    """Extract text inside angle brackets from a string.
    Args:
        input_string: The string to search for bracketed text
    Returns:
        List of strings found inside brackets, or None if no matches
    """
    matches = re.findall(r'<(.*?)>', input_string)

    if matches:
        return matches
    else:
        print("No text found inside angle brackets.")
        return None
```

The three selected models; Gemini 2.5 Pro, Claude Sonnet 4, and DeepSeek: R1 0528, were evaluated in separate runs. Each prompt was sent to every model 5 times. The final solution was determined by taking the mode (most frequent) misinformation pattern from the five responses. When two different patterns each appeared twice, both were recorded as selected solutions.

4.6.2 Experiment Evaluation Approach

Whether the solution provided by the model matches the annotator's solution is evaluated in the same way as in Chapter 3.2 *Annotation Campaign Results*, using three categories. A *perfect match* means the model selected the same misinformation pattern as the annotators, while a *partial match* means the solution is matching either in the adjective or in the noun.¹³

Technical Settings

The models were prompted via API services. *OpenRouter*¹⁴ was used, a service that provides access to models from multiple providers through one unified API, which simplifies the setup required for switching between different models. The same parameters described in Chapter 4.3 *Evaluating LLM Consistency* were used to maximize reproducibility.

¹³ This evaluation approach has limitations: fact-check reports with multiple solutions are more likely to achieve partial matches, and theoretically, an LLM could maximize performance by always proposing two solutions. However, since each prompt is processed in an independent session, the model cannot strategically adjust its responses. In traditional ML, models could overfit to exploit such metrics, but this is not a concern in our zero-shot LLM setting. The partial match category remains valuable as it provides insights into cases where models capture some aspect of the misinformation pattern correctly.

¹⁴ <https://openrouter.ai/>

Code Snippet 4.2

Invoking LLM models via OpenRouter API

```
import os
from dotenv import load_dotenv
from typing import List
from openai import OpenAI

class OpenRouterModel(BaseModel):
    def __init__(self,
                 model: str,
                 api_key_name: str = "OPENROUTER_API_KEY"
                 ):

        self.model = model
        open_router_api_key = os.environ.get(api_key_name)
        self.client = OpenAI(base_url="https://openrouter.ai/api/v1",
                             api_key=open_router_api_key)
        self.temperature = 0

    def complete(self, user_messages: List):
        response = self.client.chat.completions.create(
            model=self.model,
            temperature=self.temperature,
            messages=user_messages,
        )

        if not response.id:
            print(f"Response blocked: {response}")
            return None, response

        response_message = response.choices[0].message
        return response_message.content, response
```

Code Snippet 4.3

Constructing the request payload, obtaining LLM response

```
def get_verdict_element(fact_check_report: str, model: OpenRouterModel) -> Tuple[List, str]:
    text_to_send = template + fact_check_report
    text_to_send_api_format = get_user_message(text_to_send)
    model_response, _ = model.complete(text_to_send_api_format)
    verdict_element_list = extract_bracketed_text(model_response)

    return verdict_element_list, model_response
```

Code Snippet 4.2 shows the function used to invoke all LLM models during the main experiment. Code Snippet 4.3 demonstrates how the prompt template was combined with the fact-check report content to form the full prompt. A function in Code Snippet 4.1 is used to extract the two-word verdict argumentation element solution from the LLM's response.

Once all solutions were collected in a table in standardized form, they were evaluated against the solutions from the annotation campaign.

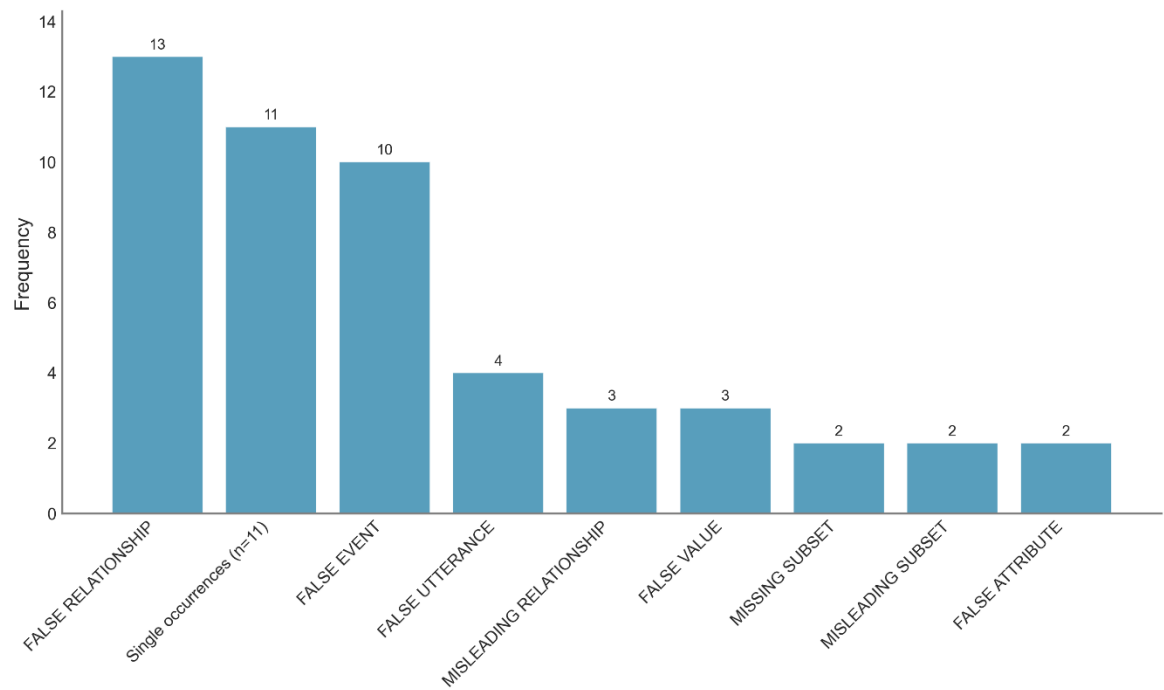
4.6.3 Experiment Results

The results of the main experiment are presented in Table 4.8. The best performing model was Gemini 2.5 Pro, with selecting the same misinformation patterns as the annotators in 46% of cases and achieving another 36% in partial matches.

Table 4.8
Automatic misinformation pattern evaluation

Model	Perfect Match	Partial Match	Different Decision
Gemini 2.5 Pro Preview 06-05	46% (23)	36% (18)	18% (9)
Claude Sonnet 4	38% (19)	32% (16)	30% (15)
DeepSeek: R1 0528	42% (21)	42% (21)	16% (8)

Figure 4.2
Distribution of misinformation patterns selected by Gemini 2.5 Pro

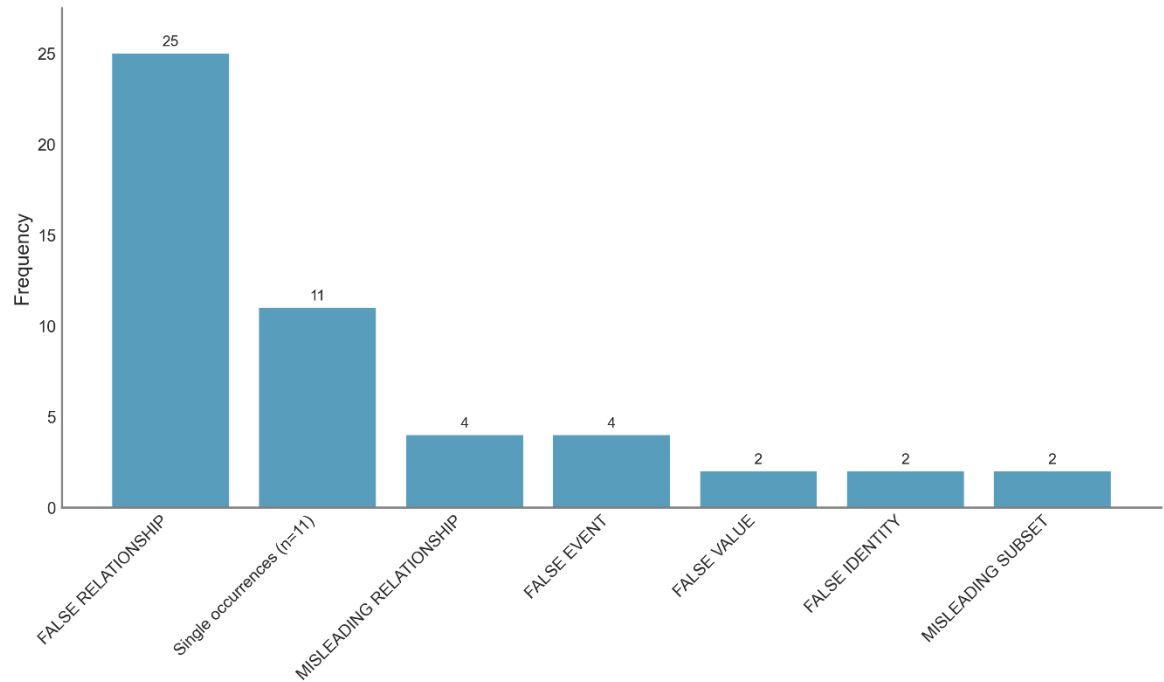


Source: Author

The distribution of misinformation patterns selected by Gemini 2.5 Pro is presented in Figure 4.3. The model chose the FALSE RELATIONSHIP verdict argumentation element in 26% of cases, followed by FALSE EVENT in 20% of cases. This can be compared to results from the annotation campaign, which are analysed in Figure 3.3 where the most selected misinformation pattern was also FALSE RELATIONSHIP, with a 23.8% selection rate, but followed by MISLEADING RELATIONSHIP selected in 11.3% of cases.

Figure 4.3

Distribution of misinformation patterns selected by DeepSeek R1

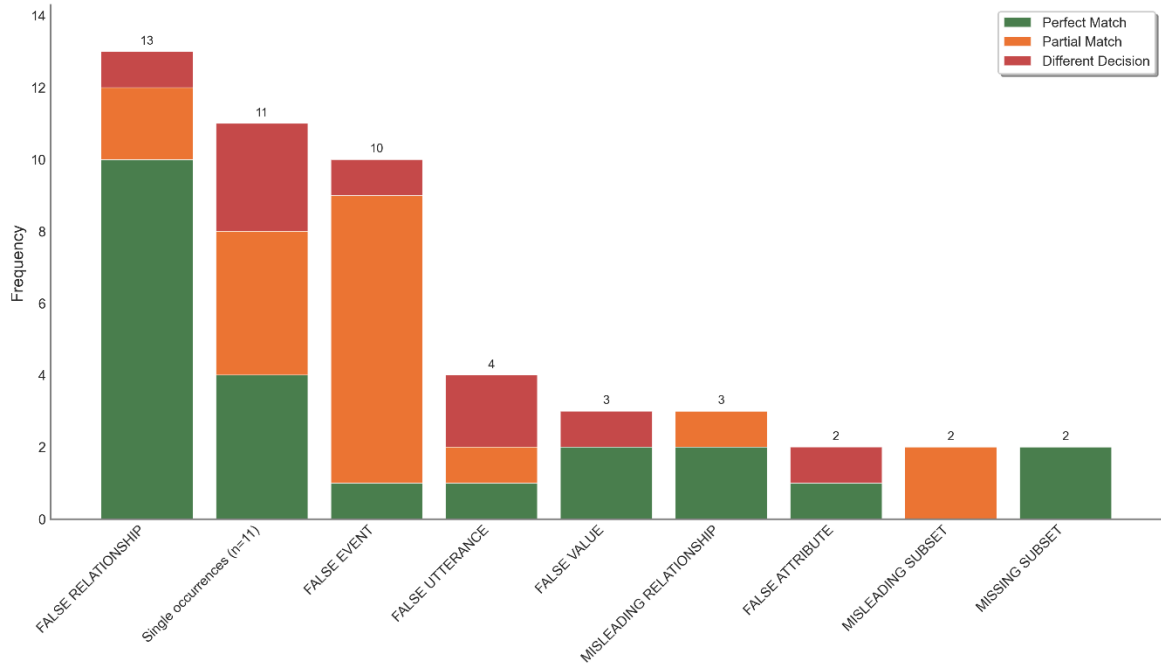


Source: Author

Even though DeepSeek R1 achieved a comparable performance to Gemini 2.5 Pro according to our classification metric presented in Table 4.8, further analysis uncovered that the model generalized worse. As Figure 4.3 demonstrates, the model defaulted to selecting the most generic misinformation pattern *FALSE RELATIONSHIP* in 50% of cases, rarely choosing more specific argumentation elements. Claude Sonnet 4 had a similar distribution pattern.

Figure 4.4

Distribution of misinformation patterns selected by Gemini 2.5 Pro with evaluation against the annotation campaign



Source: Author

Figure 4.4 extends on Figure 4.2 by including a comparison of the Gemini 2.5 Pro results to the annotation campaign. We can observe that when the model selected FALSE RELATIONSHIP as the most prominent misinformation pattern, it was correct in most of the cases. On the other hand, when selecting FALSE EVENT, it was mostly evaluated as only a partial match to the annotator's decision.

From the above stated evaluations, we can conclude the best performing model for the task of automatically finding misinformation patterns is Gemini 2.5 Pro. The experiment demonstrates that large language models can indeed automatically classify misinformation patterns when provided with fact-check reports as input.

4.6.4 Misinformation Pattern Relatedness

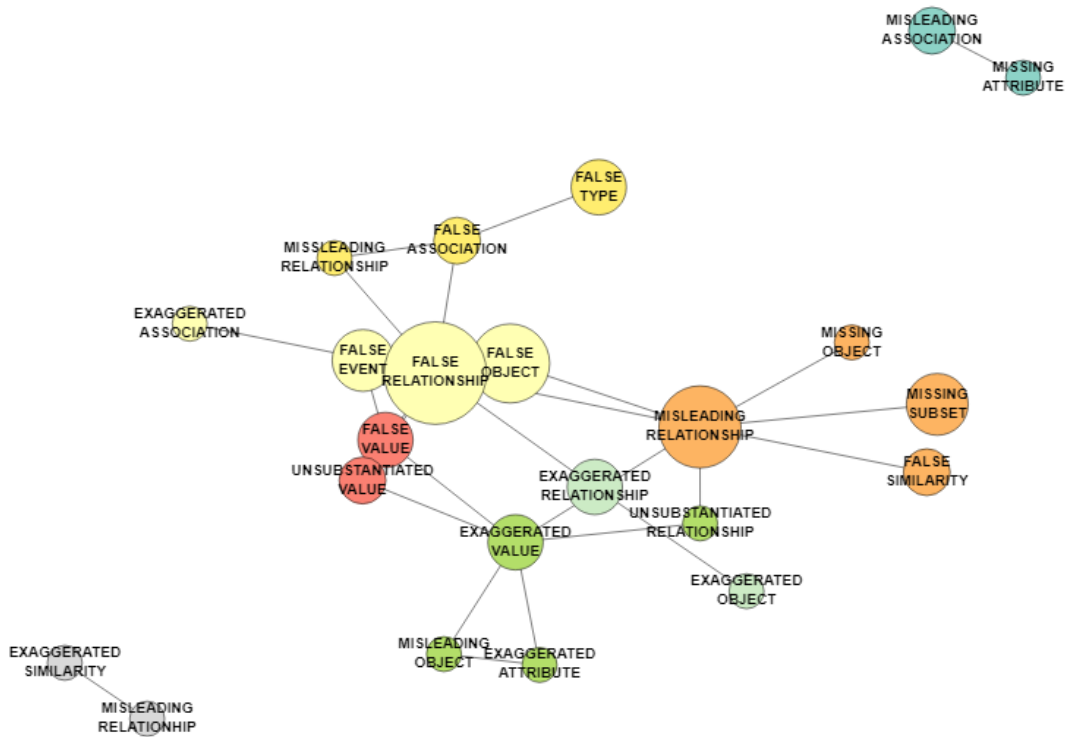
During the annotation campaign, it was observed that fact-check reports could often be modeled using multiple misinformation patterns (Table 3.2). This finding prompted me to investigate whether certain verdict argumentation elements frequently co-occur. This could mean the patterns are related in their meaning. Such relationships could guide future development of sophisticated evaluation metrics that better capture partial matches between patterns.

Figure 4.5 shows an experimental network graph that aims to map the relationships between misinformation patterns selected by the annotators during the annotation campaign. Misinformation patterns are evaluated as related when they are proposed together as multiple possible solutions to a single fact-check report. The network is

constructed by calculating co-occurrence frequencies between pattern pairs and applying the Louvain community detection algorithm to identify clusters of semantically related misinformation strategies, with node sizes reflecting pattern frequency and colors indicating community membership.

Figure 4.5

Network graph of misinformation patterns selected in the annotation campaign



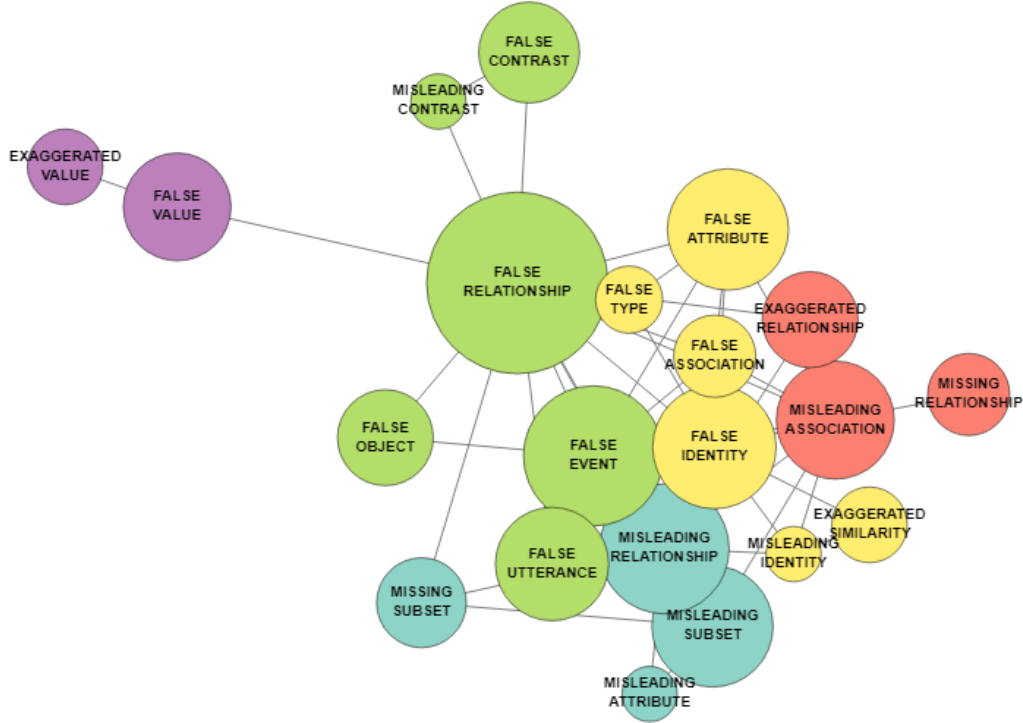
Source: Author

Multiple related communities of patterns can be observed. Namely, the most prominent community consists of FALSE RELATIONSHIP, FALSE OBJECT, FALSE EVENT, and EXAGGERATED ASSOCIATION (Figure 4.5).

From the way the main experiment using three different LLMs was conducted, each evaluating one fact-check report five times, we have obtained 15 different proposals for each report. From this data, we can construct a network graph the same way we did in Figure 4.5. This network is presented in Figure 4.6.

Figure 4.6

Network graph of misinformation patterns selected by LLMs in the main experiment



Source: Author

Since we have significantly more proposals for each report compared to the annotation campaign, several methodological adjustments were made. The node sizes representing misinformation pattern frequency are weighted logarithmically, and the Louvain algorithm is used to construct the communities. The increased data density from the LLM experiment (15 proposals per report versus typically 2-3 from human annotators) required raising the co-occurrence weight threshold to 2 and employing the Kamada-Kawai layout algorithm for more stable positioning of the denser network structure.

We can observe multiple distinct communities forming in Figure 4.6. The most prominent community being centered around the patterns *FALSE RELATIONSHIP* and *FALSE EVENT*. This graph network demonstrates that LLM-based automatic classification can capture multiple possible interpretations of a single fact-check report. Such diversity in classification outcomes potentially strengthens our analysis by providing more comprehensive argumentation coverage across different viewpoints.

5 Automatic Explanation of Misinformation Patterns

The goal of this chapter is to extend the work proposed in Chapter 4 *Automatic Classification of Misinformation Patterns* by focusing on explaining why the LLM selected the misinformation pattern it did for a given fact-check report. The aim is not only to provide textual reasoning for the decision but also to propose an automatic approach to display the argumentation using a entity graph representation. The graph can be used as an overview of the argumentation present in a given fact-check report, making it more accessible to readers.

5.1 Automatic Explanation Generation

The prompt for automatically detecting misinformation patterns already asks the LLM to provide its reasoning before outputting the final decision (Prompt 4.2). An example of output generated by Gemini 2.5 Pro when queried to analyse the "fire drone" fact-check report example is presented in LLM output 5.1.

LLM output 5.1

Gemini 2.5 Pro analysis of the "fire drone" fact-check report

```
B-Entities:
The video
The aircraft in the video (claimed to be a drone, is a helicopter)
The act of setting fires
The West Coast wildfires
Twitter, Google, and YouTube

B-Relationship 1: is the cause of
This relationship is implied by the claim's context.
Subject: The act of setting fires (shown in the video)
Object: The West Coast wildfires
B-Attributes:
Purpose -> B-Value: Malicious arson (implied)

B-Relationship 2: is suppressed by
This relationship is explicitly stated in the claim.
Subject: The video
Object: Twitter, Google, and YouTube

The fact-check report refutes the implied purpose of the fire-setting shown in the video. The claim frames it as a malicious act of arson causing the wildfires. However, the report clarifies that the video actually shows a "controlled burn," which is a standard firefighting tactic used to prevent larger fires. The core deception lies in misrepresenting the purpose and context of the action in the video, thereby creating a false causal link to the ongoing wildfires.

Verdict argumentation element: <FALSE RELATIONSHIP> connecting the act of setting fires to the West Coast wildfires.
```

We can observe that the model produced a comparable reasoning and the same verdict argumentation element as our analysis in the Section *Example 2: Fire Drone Video*. The generated reasoning explanations act as an input for the next phase: automatically generating argumentation graphs.

5.2 Automatic Entity Graph Generation

My original idea was to extend the PURO modeler, which is already available online and supports importing argumentation graphs using a specific format. Nevertheless, converting textual argumentation into the hard-coded format proved too challenging. Therefore, I settled for an approach that does not require strict adherence to all PURO modeling language rules.

I propose a prompt blueprint that asks the LLM to analyse its previous argumentation and design an argumentation graph based on simplified description of the PURO language rules.¹⁵ The model is asked to output Python code using specified libraries and approaches to plot the argumentation graph. Prompt 5.1 has been developed using a similar iterative approach as described in Chapter 4.5 Prompt Refinement.

Prompt 5.1

Prompt blueprint - visualizing the LLM argumentation with entity graphs

Goal: Write a Python script to create a visual representation of the provided argumentation which analyses a fact-check report.

Original Claim:
<The original claim>

Report analysis:
<Output from previous step>

SECTION 2: GRAPH BUILDING RULES AND TASK

A. Component Visuals:

Component Type	Visual
B-object	Blue Rectangle
B-relationship	Green Rhombus
B-Attribute	Orange Rectangle
B-value	Yellow Rectangle (quantitative values)
Verdict-or-Note	Grey Rectangle

B. Graph Construction Rules:

Connection Flow: Connections must follow a strict hierarchy to ensure clarity:
B-object nodes can only connect to B-relationship nodes or B-attributes. They can never connect directly to another B-object.
B-relationship nodes are the central connectors. They can connect B-object nodes only.
Verdict nodes or notes connect to the specific component they are refuting.

C. Task:

¹⁵ A simplified description of the PURO modeling language is used for the sake of keeping the task manageable for LLMs

Part 1: State Your Plan
Before writing any code, begin your response by reiterating your plan. This plan must include:

A list of all B-object nodes you will create.

Important: You are allowed to change the original lists of objects and relationship for the sake of clarity (possibly dropping entities) and following the mandates. Only thing which shall not be changed is the verdict element. (You are allowed to use notes which point to other problematic relationships). Focus on explainability - a simpler graph is better than cluttered.

Part 2: Write the Script
After stating your plan, produce a single, complete Python script using networkx and matplotlib that executes your plan. The script must:

Use the exact shapes and colors specified in the Component Visuals table.

Use a layout algorithm (e.g., `spring_layout`) and manual adjustments to create a highly readable graph with no overlapping nodes or labels.

Label all nodes and arrows clearly based on your plan.

Use directed arrows for all connections.

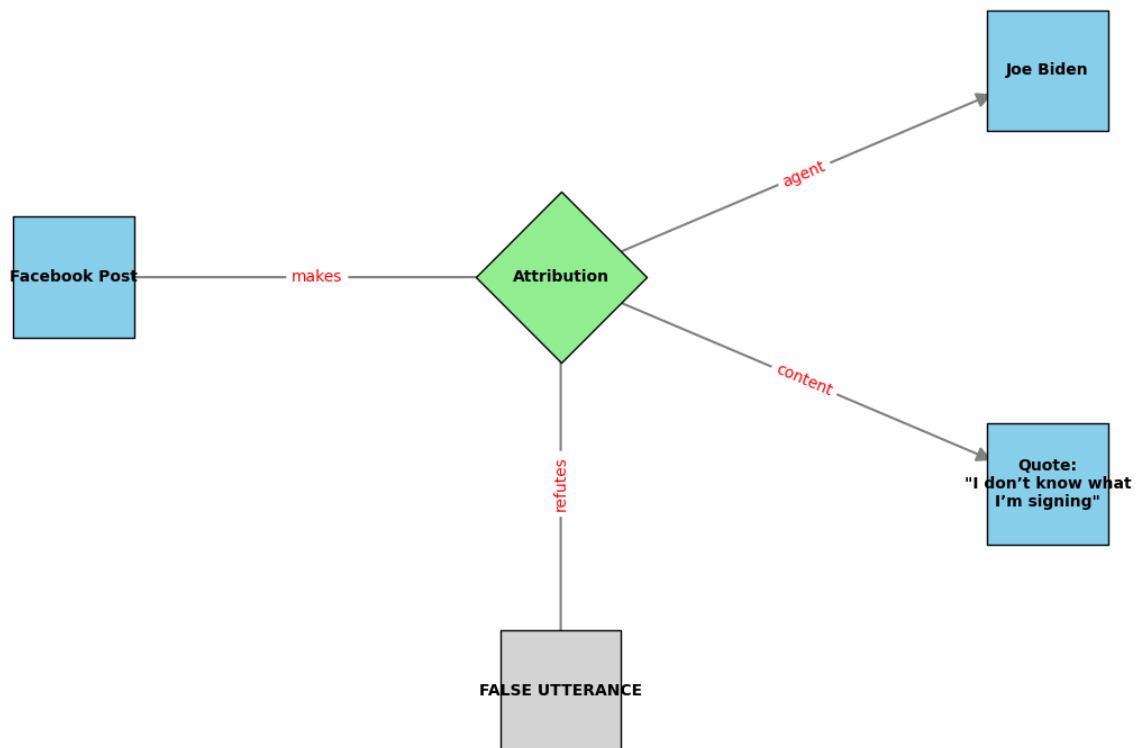
5.2.1 Examples of LLM-Generated Graphs

The following section presents examples of diagrams automatically generated by an LLM. All diagrams are created using a two-step approach. In the first step, the LLM is prompted with Prompt 4.2 to choose the verdict argumentation element and generate an overview and explanation of its reasoning. In the second step, the output from step 1 is combined with Prompt 5.1 to form the input. The model outputs executable Python code, which is executed in a secure environment (running the code directly on a personal computer could pose safety risks, such as directory deletion). The code produces an image of the graph using the matplotlib library - this is the final output. The following experiments are all conducted using Gemini 2.5 Pro. While other LLMs were also evaluated, Gemini significantly outperformed other models for this task of generating Python code.

Figure 5.1 presents the entity graph created automatically based on the *document signing utterance* fact-check report. It can be directly compared to the manually created graph from Figure 1.1.

Figure 5.1

Argumentation diagram of the second example fact-check - created automatically by Gemini 2.5 Pro



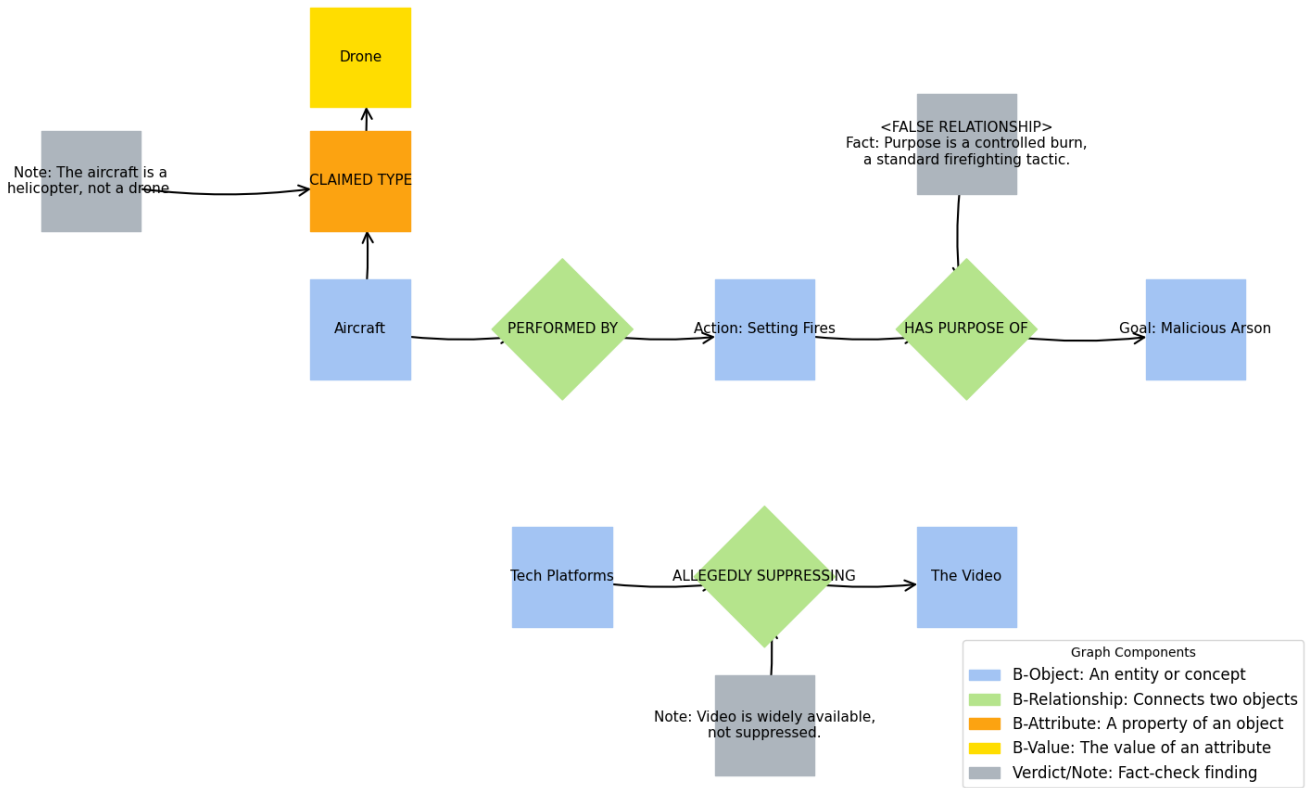
Source: Author

The model created an additional entity "Facebook Posts" and instead of connecting the verdict argumentation element FALSE UTTERANCE to the quote, it connected it to the Attribution relationship. Nevertheless, the generated graph is remarkably similar to the human-created version.

Figure 5.2 presents an entity graph generated by Gemini 2.5 Pro based on the fire drone fact-check report. This example is discussed in *Example 2: Fire Drone Video*, and the graph can be compared to the human annotator's modeling presented in Figure 1.2.

Figure 5.2

Argumentation diagram of the fire drone fact-check report - created automatically by Gemini 2.5 Pro



Source: Author

The initial observation is the visual style of the diagram, which varies for each iteration since the approach for plotting the diagram in Python code is not specifically defined. The LLM modeled the fact-check report in more detail compared to Figure 1.2. The argumentation is split into two sections: the upper part analyses the alleged real-world event of a fire drone setting fires. In contrast, the lower section describes the alleged suppression of the video by tech platforms. While the proposed argumentation closely resembles the annotator's approach, the FALSE RELATIONSHIP verdict argumentation element is attributed to the purpose of setting fires by the drone, whereas the annotator attributed FALSE RELATIONSHIP to the alleged suppression of the video. In this case, both possibilities are valid, and the diagram could display both verdict argumentation elements.

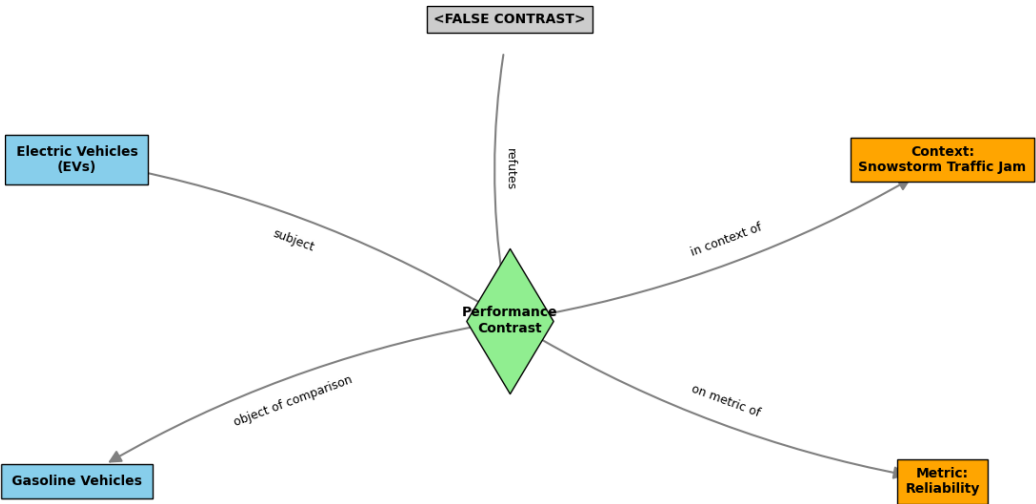
Possible improvements to the graph in Figure 5.2 include, for example, changing the relationship *Aircraft -> Performed by -> Action: Setting fires* to *Aircraft -> Performs -> Action: Setting Fires*.

The annotators chose to model this report with the *UNSUBSTANTIATED EVENT* verdict argumentation element, pointing out that the event of electric vehicles losing battery capacity while standing in a snowstorm is not based on evidence.

Figure 5.3 shows an automatically generated entity graph for a fact-check report that was evaluated differently by the LLMs and the annotators. The original claim reads *Electric vehicles would not have fared well in the Virginia snowstorm traffic jam*.¹⁶ The reasoning in the fact-check report argues that there is no evidence that electric vehicles would lose their battery capacity while standing still in a traffic jam during a snowstorm; therefore, the claim was rated as false. The annotators chose to model this report with the *UNSUBSTANTIATED EVENT* verdict argumentation element, pointing out that the event of electric vehicles losing battery capacity while standing in a snowstorm is not based on evidence.

Figure 5.3

Argumentation diagram of an "electric vehicles in a snowstorm" fact-check - created automatically by Gemini 2.5 Pro



Source: Author

The LLM model, on the other hand, selected the verdict argumentation element FALSE CONTRAST, comparing the performance of gasoline vehicles to electric vehicles (Figure 5.3). We can also observe that two attribute elements were created: one describing the context of the situation, and another specifying that the performance contrast in question relates to reliability. However, in this case, the usage of the attribute elements could be improved, since the "Context: Snowstorm Traffic jam" element is not truly an attribute of the "Performance Contrast" relationship.

¹⁶ Full fact-check report available at <https://www.politifact.com/factchecks/2022/jan/07/facebook-posts/theres-no-evidence-electric-vehicles-fare-worse-ga/>

Although this specific scenario was rated as a *different decision* in the evaluation of the main experiment, an argument could be made for both the human annotator's solution and the solution generated by Gemini 2.5 Pro. The discussion of which possibility is more suitable must be grounded in the rules of the underlying modeling language. Personally, I would favor the automated solution, since the complete fact-check report cites the original Facebook post, which explicitly compares electric vehicles to gasoline and diesel vehicles. The post attempts to establish a contrast between how well EVs would fare in a snowstorm traffic jam compared to gasoline and diesel cars. However, the reasoning in the fact-check report refutes this difference, hence the argumentation can be appropriately modeled using the FALSE CONTRAST misinformation pattern.

6 Conclusions

The primary objective of this thesis was to investigate whether large language models can detect misinformation patterns in fact-check reports. Additional research questions addressed how to make LLMs' decisions more interpretable by automatically modeling the argumentation using entity graphs, and whether the modeling is consistent and reliable.

First, a dataset of 50 fact-check reports was collected from PolitiFact to serve as a source dataset of fact-checked claims with potentially rich argumentation patterns. The reports were annotated by five researchers in an annotation campaign. Each report was assigned its core misinformation patterns, key argumentation snippets were selected, and a short explanation of the annotator's reasoning was constructed.

To evaluate the consistency of LLMs in selecting verdict argumentation elements, an experiment with seven prominent LLMs was conducted. Each model was prompted 5 times with a baseline prompt consisting of simple instructions on detecting misinformation patterns based on a pre-defined catalog, together with the fact-check report text. The results showed that model consistency varies based on both the models' capabilities and the complexity of the fact-check report being modeled. Less capable LLMs, such as Gemini 2.5 Flash, struggled to follow instructions when analysing more complex fact-checks, while Gemini 2.5 Pro consistently outperformed all other models.

To evaluate what makes a fact-check complex, three parameters were proposed: entity density, contextual requirements, and clarity of the verdict. Future work could further refine these parameters and propose an automated method to calculate them, thereby forming a fact-check report complexity metric that indicates how well LLMs or other automated methods can model the report.

The main experiment developed a prompt to detect the most prominent misinformation pattern automatically. All 50 fact-check reports were evaluated using 3 LLMs (Gemini 2.5 Pro, Deepseek R1, and Claude Sonnet 4) and compared to results from the annotation campaign. Gemini 2.5 Pro clearly managed to model argumentation the best, achieving agreement with annotators on 46% of fact-check reports, with an additional 36% evaluated as partial matches.

Since one fact-check report can be modeled in multiple ways (multiple misinformation patterns can be valid), the results from the main experiments were used to construct a network diagram providing an overview of how different misinformation patterns relate to each other. This assumes that when multiple solutions are proposed for a single fact-check report, the proposed misinformation patterns are related.

As the results from the main experiment may be difficult to interpret (e.g., why the model selected a different solution compared to the annotators), a method was proposed to automatically create an argumentation entity graph inspired by the PURO modeling language. The solution leverages the LLMs' ability to generate executable code. In the

proposed 2-shot approach, the LLM is first prompted to explain the fact-check report and select the most prominent misinformation pattern. The output from the first step is used to generate Python code that displays the argumentation graph. This enabled an explainable comparison of the model's reasoning against the annotator's reasoning.

Analysis of selected fact-checked reports demonstrated the approach's potential: in most cases, LLMs produced similar modeling of fact-check reports compared to annotators. However, a similar limitation to other experiments was observed: the more complex the fact-check reports, the more difficult it is to accurately model argumentation, with LLMs being susceptible to pitfalls like focusing on less important details, struggling with very long contexts, or falling for prompt injection attacks.

While the results of the experiments in this thesis are promising, it is crucial to be aware of the limitations of using LLMs, especially for a critical field such as fact-checking. LLMs are famously unreliable "by design." They often inherit biases from their training data and are susceptible to prompt injection attacks and other methods of "fooling" the models, which can be especially dangerous if the bad actor is aware their posts are being fact-checked automatically. For these reasons, the proposed approach, and ultimately LLMs in general, should be used as a complement to human fact-checkers. In other words, any output generated by an LLM should be carefully manually reviewed by us, humans.

The research questions of the thesis have been successfully answered. The proposed approach can be further extended and used either as an interpretability layer for fact-checked claims or expanded into an automatic method for evaluating the quality of fact-check reports.

List of references

- Advanced usage—OpenAI API*. (n.d.). Retrieved 17 June 2025, from <https://platform.openai.com>
- Allahyari, M., Pouriyeh, S., Assefi, M., Safaei, S., Trippe, E. D., Gutierrez, J. B., & Kochut, K. (2017). *Text Summarization Techniques: A Brief Survey* (No. arXiv:1707.02268). arXiv. <https://doi.org/10.48550/arXiv.1707.02268>
- Bahdanau, D., Cho, K., & Bengio, Y. (2016). *Neural Machine Translation by Jointly Learning to Align and Translate* (No. arXiv:1409.0473). arXiv. <https://doi.org/10.48550/arXiv.1409.0473>
- Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhang, H., Zhu, B., Jordan, M., Gonzalez, J. E., & Stoica, I. (2024). *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference* (No. arXiv:2403.04132). arXiv. <https://doi.org/10.48550/arXiv.2403.04132>
- Dudáš, M., Svátek, V., Vacura, M., & Zamazal, O. (2016). *Starting Ontology Development by Visually Modeling an Example Situation—A User Study*.
- Ferreira Caceres, M. M., Sosa, J. P., Lawrence, J. A., Sestacovschi, C., Tidd-Johnson, A., Rasool, M. H. U., Gadamidi, V. K., Ozair, S., Pandav, K., Cuevas-Lou, C., Parrish, M., Rodriguez, I., & Fernandez, J. P. (2022). The impact of misinformation on the COVID-19 pandemic. *AIMS Public Health*, 9(2), 262–277. <https://doi.org/10.3934/publichealth.2022018>
- Guo, Z., Schlichtkrull, M., & Vlachos, A. (2022). A Survey on Automated Fact-Checking. *Transactions of the Association for Computational Linguistics*, 10, 178–206. https://doi.org/10.1162/tacl_a_00454
- Haniková, K. (2025). Explaining Misleading Claims Using Graphs of Entities. In A. Meroño Peñuela, O. Corcho, P. Groth, E. Simperl, V. Tamma, A. G. Nuzzolese, M. Poveda-Villalón, M. Sabou, V. Presutti, I. Celino, A. Revenko, J. Raad, B. Sartini, & P. Lisena

- (Eds.), *The Semantic Web: ESWC 2024 Satellite Events* (pp. 94–103). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-78955-7_10
- Haniková, K., Chudán, D., Svátek, V., Vajdečka, P., Troncy, R., Vencovský, F., & Syrovátková, J. (2024). Towards Fact-check Summarization Leveraging on Argumentation Elements Tied to Entity Graphs. *Companion Proceedings of the ACM Web Conference 2024*, 1473–1481. <https://doi.org/10.1145/3589335.3651914>
- Hassan, N., Arslan, F., Li, C., & Tremayne, M. (2017). Toward Automated Fact-Checking: Detecting Check-worthy Factual Claims by ClaimBuster. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1803–1812. <https://doi.org/10.1145/3097983.3098131>
- Jones, C. R., & Bergen, B. K. (2025). *Large Language Models Pass the Turing Test* (No. arXiv:2503.23674; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2503.23674>
- Klishevich, E., Denisov-Blanch, Y., Obstbaum, S., Ciobanu, I., & Kosinski, M. (2025). *Measuring Determinism in Large Language Models for Software Code Review* (No. arXiv:2502.20747; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2502.20747>
- Konstantinovskiy, L., Price, O., Babakar, M., & Zubiaga, A. (2020). *Towards Automated Factchecking: Developing an Annotation Schema and Benchmark for Consistent Automated Claim Detection* (No. arXiv:1809.08193). arXiv. <https://doi.org/10.48550/arXiv.1809.08193>
- Luhn, H. P. (1958). The Automatic Creation of Literature Abstracts. *IBM Journal of Research and Development*, 2(2), 159–165. <https://doi.org/10.1147/rd.22.0159>
- Nie, Y., Chen, H., & Bansal, M. (2018). *Combining Fact Extraction and Verification with Neural Semantic Matching Networks* (No. arXiv:1811.07039). arXiv. <https://doi.org/10.48550/arXiv.1811.07039>
- Panchendrarajan, R., & Zubiaga, A. (2024). Claim Detection for Automated Fact-checking: A Survey on Monolingual, Multilingual and Cross-Lingual Research. *Natural*

- Language Processing Journal*, 7, 100066.
<https://doi.org/10.1016/j.nlp.2024.100066>
- Schmidt, R. M. (2019). *Recurrent Neural Networks (RNNs): A gentle Introduction and Overview* (No. arXiv:1912.05911). arXiv.
<https://doi.org/10.48550/arXiv.1912.05911>
- Singh, S., Nan, Y., Wang, A., D'Souza, D., Kapoor, S., Üstün, A., Koyejo, S., Deng, Y., Longpre, S., Smith, N. A., Ermiş, B., Fadaee, M., & Hooker, S. (2025). *The Leaderboard Illusion* (No. arXiv:2504.20879). arXiv.
<https://doi.org/10.48550/arXiv.2504.20879>
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). *FEVER: A large-scale dataset for Fact Extraction and VERification* (No. arXiv:1803.05355). arXiv.
<https://doi.org/10.48550/arXiv.1803.05355>
- Vajdecka, P., Svatek, V., & Vita, M. (2023). *A Novel Approach to Abstractive Summarization Based on LOF, Sentence-BERT and T5 – with Fact Checking Use Case* (SSRN Scholarly Paper No. 4493592). Social Science Research Network.
<https://doi.org/10.2139/ssrn.4493592>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention Is All You Need* (No. arXiv:1706.03762). arXiv.
<https://doi.org/10.48550/arXiv.1706.03762>
- Vlachos, A., & Riedel, S. (2014). Fact Checking: Task definition and dataset construction. In C. Danescu-Niculescu-Mizil, J. Eisenstein, K. McKeown, & N. A. Smith (Eds.), *Proceedings of the ACL 2014 Workshop on Language Technologies and Computational Social Science* (pp. 18–22). Association for Computational Linguistics. <https://doi.org/10.3115/v1/W14-2508>
- Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science*, 359(6380), 1146–1151. <https://doi.org/10.1126/science.aap9559>
- World Bank Open Data*. (n.d.). World Bank Open Data. Retrieved 7 April 2025, from <https://data.worldbank.org>

- Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Kaiser, Ł., Gouws, S., Kato, Y., Kudo, T., Kazawa, H., ... Dean, J. (2016). *Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation* (No. arXiv:1609.08144). arXiv. <https://doi.org/10.48550/arXiv.1609.08144>
- Zhang, X., Cao, J., Li, X., Sheng, Q., Zhong, L., & Shu, K. (2021). Mining Dual Emotion for Fake News Detection. *Proceedings of the Web Conference 2021*, 3465–3476. <https://doi.org/10.1145/3442381.3450004>
- Zhang, Y., Jin, H., Meng, D., Wang, J., & Tan, J. (2025). *A Comprehensive Survey on Process-Oriented Automatic Text Summarization with Exploration of LLM-Based Methods* (No. arXiv:2403.02901). arXiv. <https://doi.org/10.48550/arXiv.2403.02901>

List of Tools

- For citations and management of bibliography, the Zotero citation manager has been used, available at <https://www.zotero.org/>.
- For language corrections and proofing, AI tools were leveraged.
 - Grammarly, available at <https://www.grammarly.com/>
 - Claude 4 Opus, available at <https://claude.ai/> and an API service available at <https://openrouter.ai/anthropic/claude-opus-4>
- Claude 4 Opus was also discussed in a collaborative manner to design the code to make Chapter 4.6.4 *Misinformation Pattern Relatedness* possible, since the technical approach is outside of the scope of this thesis.

Appendices

Appendix A: Politifact Dataset

The file *politifact_climate_dataset.xlsx* contains a dataset of 1032 climate-related fact-check reports scraped from [politifact.com](https://www.politifact.com). A subset of the reports

Appendix B: Automation Misinformation Classification Experiment results

The file *misinformation_pattern_detection_main_experiment.xlsx* contains a detailed breakdown of the main experiment, including all prompts used and all LLM outputs.

Appendix C: Source Code of Generated Entity Graphs

The files *document_signing_entity_graph.py*, *ev_cars_entity_graph.py*, and *fire_drone_entity_graph.py* contain the Python source code of the automatically generated graphs, which were discussed in Chapter 5.2.1.

Appendix D: Github Repository Snapshot

The archive *fact-check-modeling.zip* is a snapshot of the Github repository, which contains source code for this thesis and data used in this thesis. The live version of the repository is available at <https://github.com/Rigos0/fact-check-modeling>

The structure of the zip archive includes:

- Directory *data* with source datasets and experiment results
- Directory *src* with the source code of automated tests
- The most recent version of the catalogue of misinformation patterns